

Project Final Report

Single Cycle Processor & Pipelined Processor

Table of Contents

1) Design and implementation	3
Single Cycle Processor.....	3
Design.....	3
ALU Design.....	3
Comparison Design	5
Comparison Inverter Design.....	6
Register File	6
RegFile Design.....	7
Datapath.....	8
Control Signals Table	13
Pipelined Processor.....	19
Procedures for pipelining.....	19
Data Hazards.....	20
Control Hazards	22
2) Simulation and testing	24
Bubble Sort.....	25
Machine code	25
Bubble Sort Testing.....	26
Counter Program (counting the number of 1s in a register).....	27
Machine code	27
Testing	27
ALU	28
Register File	42
3) Teamwork:.....	43
Work Distribution	43
Time Chart.....	43
4) Conclusion	44

1) Design and implementation

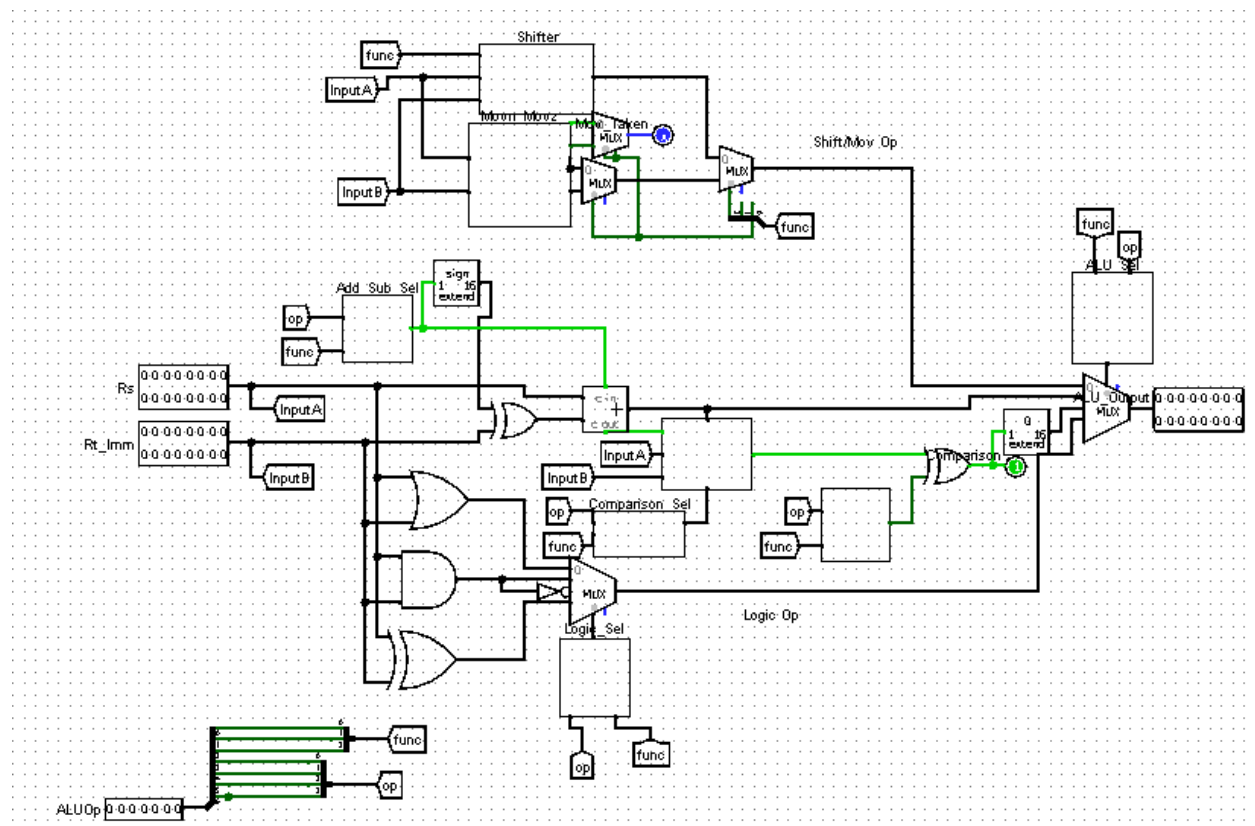
Single Cycle Processor

Design

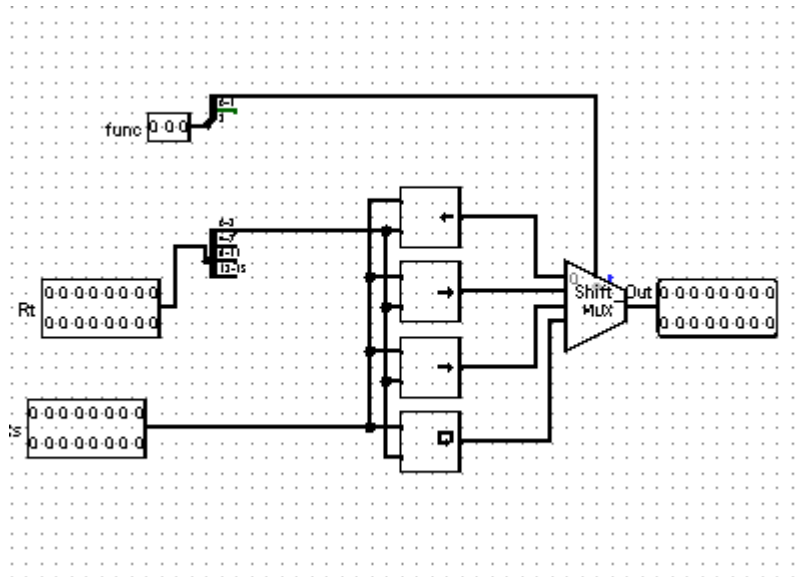
For the design, it was similar to the ones in the labs but with minor differences. For the control signals, we did not use a decoder or a ROM to generate the control signals. Instead, we used logic gates and Boolean expressions derived from Truth Tables and K-Maps for the control, which was more tedious than using a decoder or a ROM. This will be more evident during the control signals part of this section.

For the ALU, we designed a multi-stage ALU. Instead of generating a big multiplexer at the end that selects from all operations, we made it into 4 smaller stages (4 MUXes) and then a larger MUX (4 by 1) that chooses the final output.

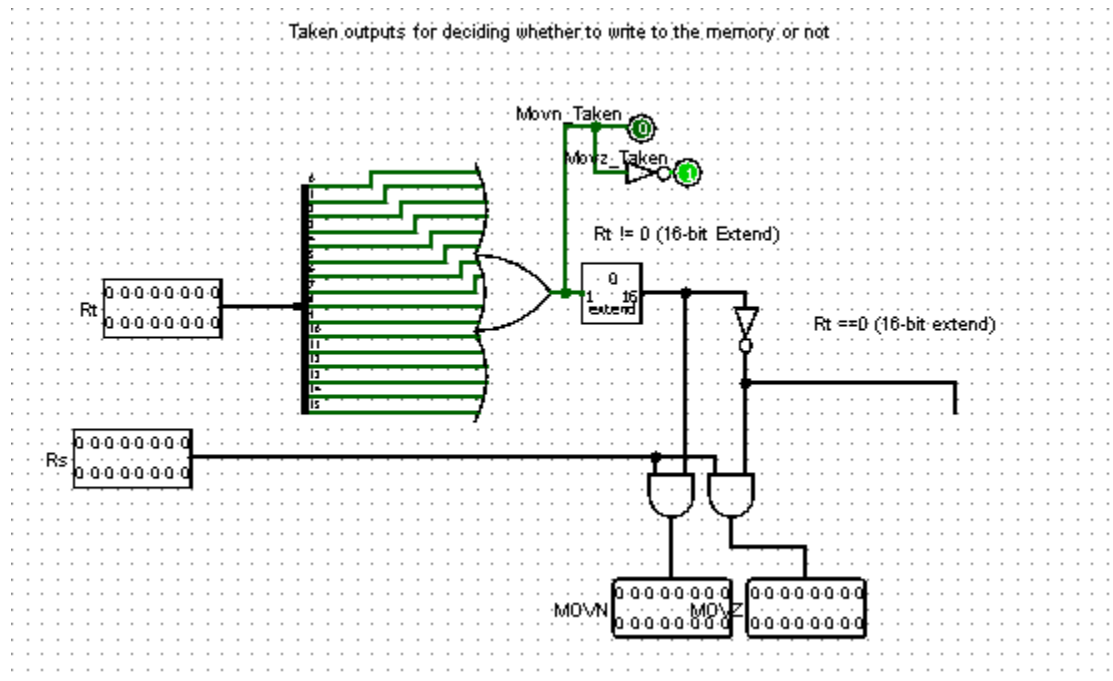
ALU Design



Shifters: We used built in shifters from Logisim that can do SLL, SRL, SRA, and ROR



MOVN_MOVZ: A circuit that determines whether the condition is satisfied or not and outputs the contents of the other register.

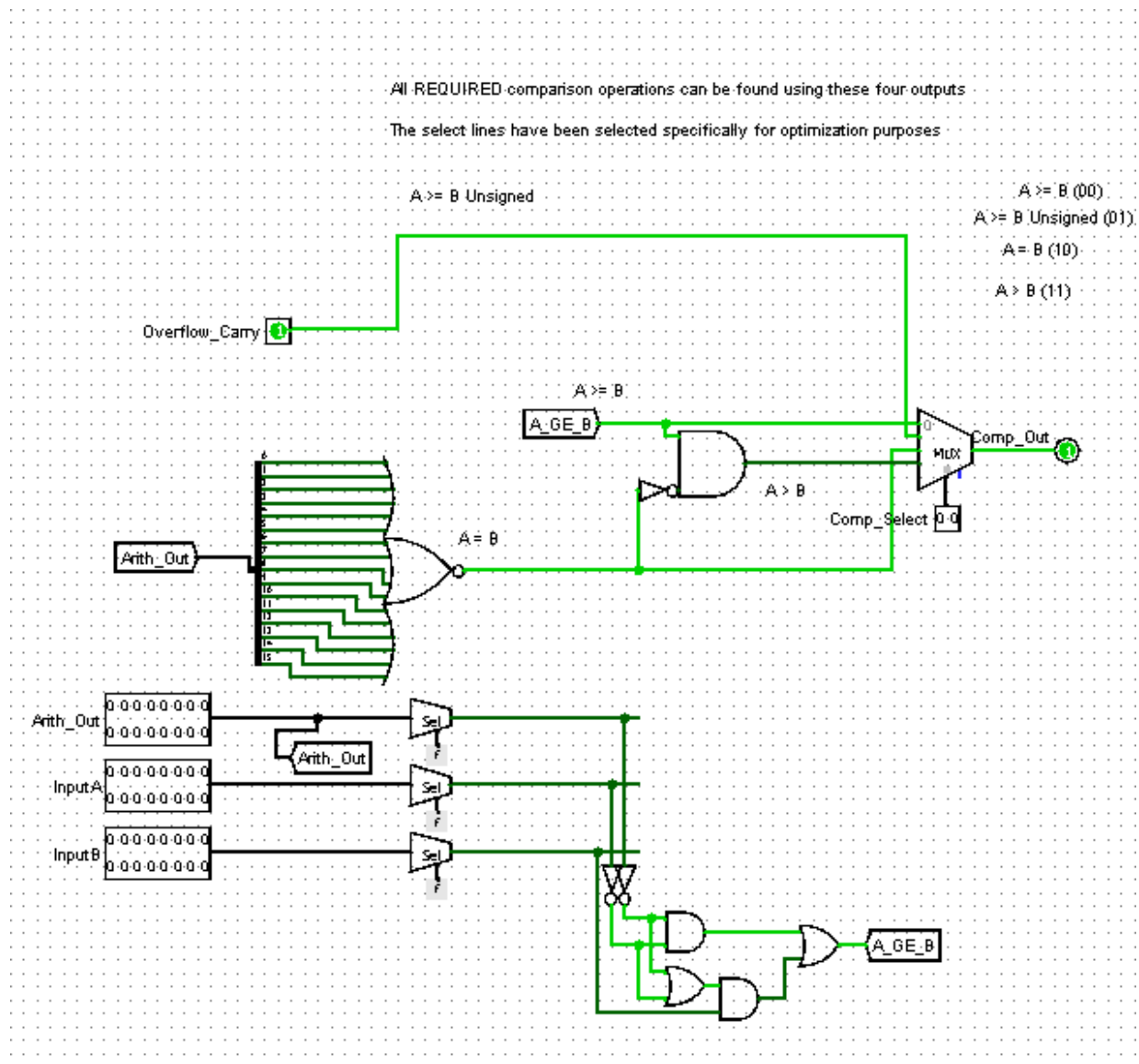


Arithmetic: As shown in the ALU figure, the adders determine the output of arithmetic operations (Addition and subtraction)

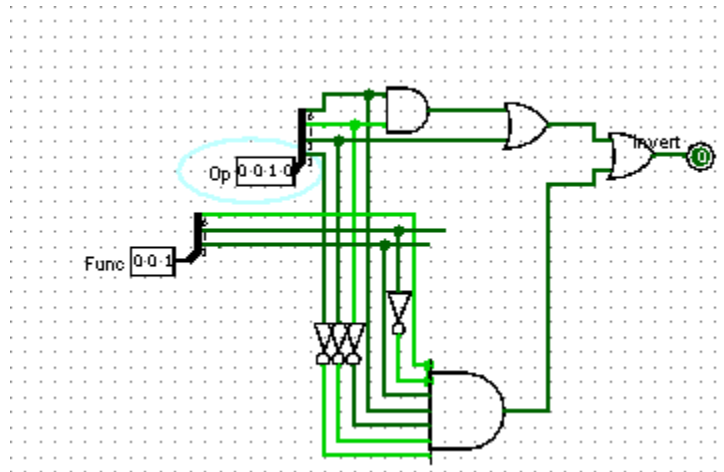
Comparison: The comparison part takes the output of the arithmetic as an input, along with the ALU inputs. It determines 4 signals ($A \geq B$ 00 , $A \geq B$ unsigned 01, $A == B$ 10, $A > B$ 11). All other comparison operations can be found by inverting the output of one of these 4 operations. Therefore, another circuit was designed to determine whether to invert the output or

not to generate other signals such as $A < B$, $A \leq B$, $A \neq B$. The output of the comparator part of the ALU is the output of comparator circuit XORed with comparator inverter circuit output.

Comparison Design



Comparison Inverter Design



Logic: The logic part is one of the simplest, the output of 4 logic gates is decided using a control signal.

Register File

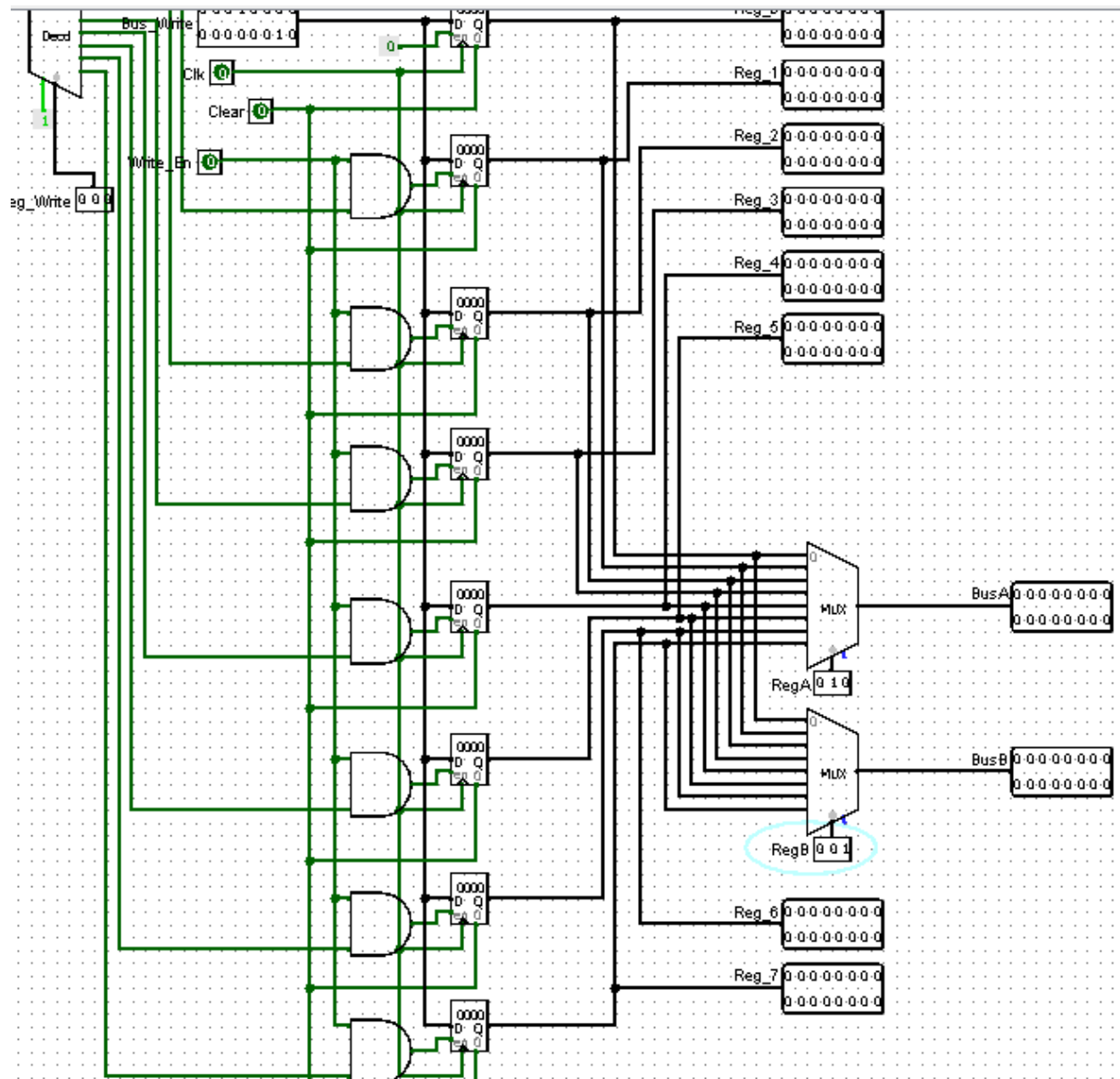
The register file takes RegA and RegB inputs that range from (0 to 7) to read from and RegW (0 to 7) to write to.

Keep in mind that Reg0 is HARDWIRED to zero, which means even if RegW == 0, the register contents will not change.

The write input of each register is on when RegW is the same as the number of that register AND Write_Enb of the Reg_File is 1.

The output BusA and BusB is taken using a decoder.

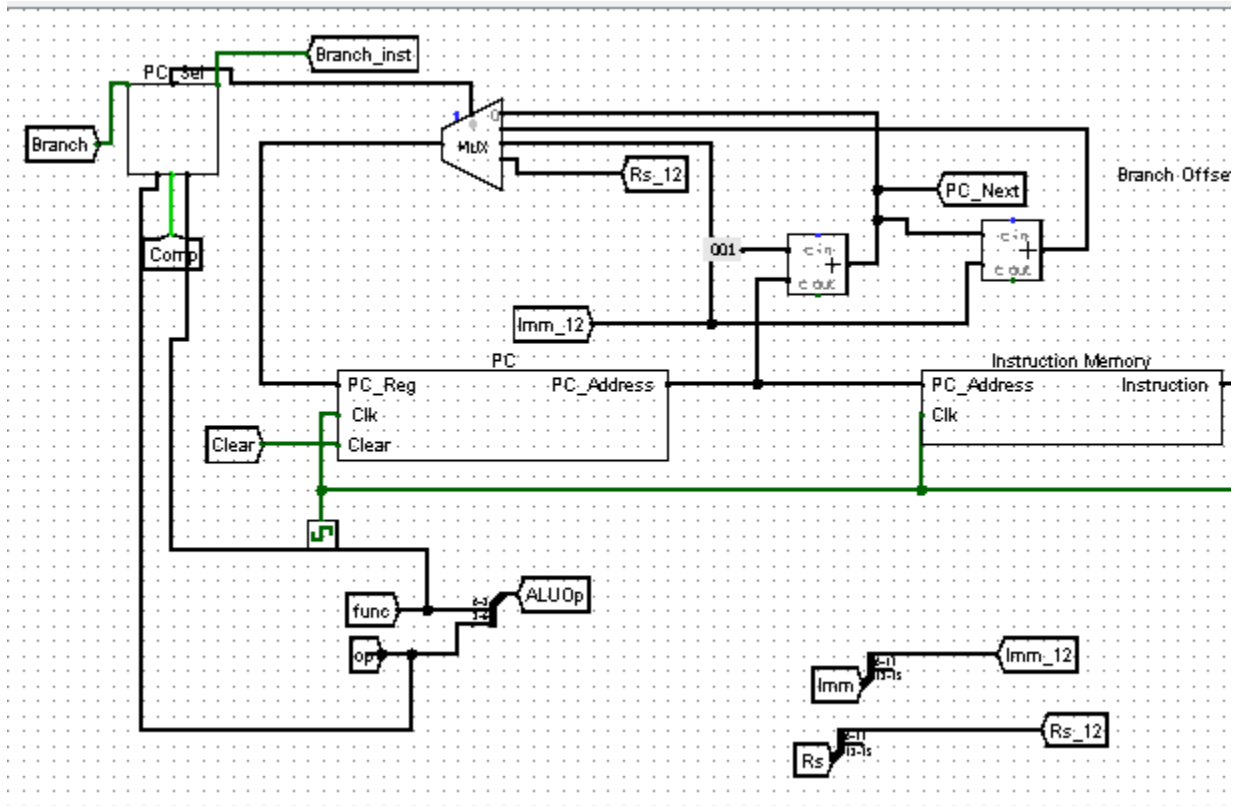
RegFile Design



OTHER CPU COMPONENTS WILL BE DISCUSSED IN THE DATAPATH AND CONTROL PART OF THE CPU

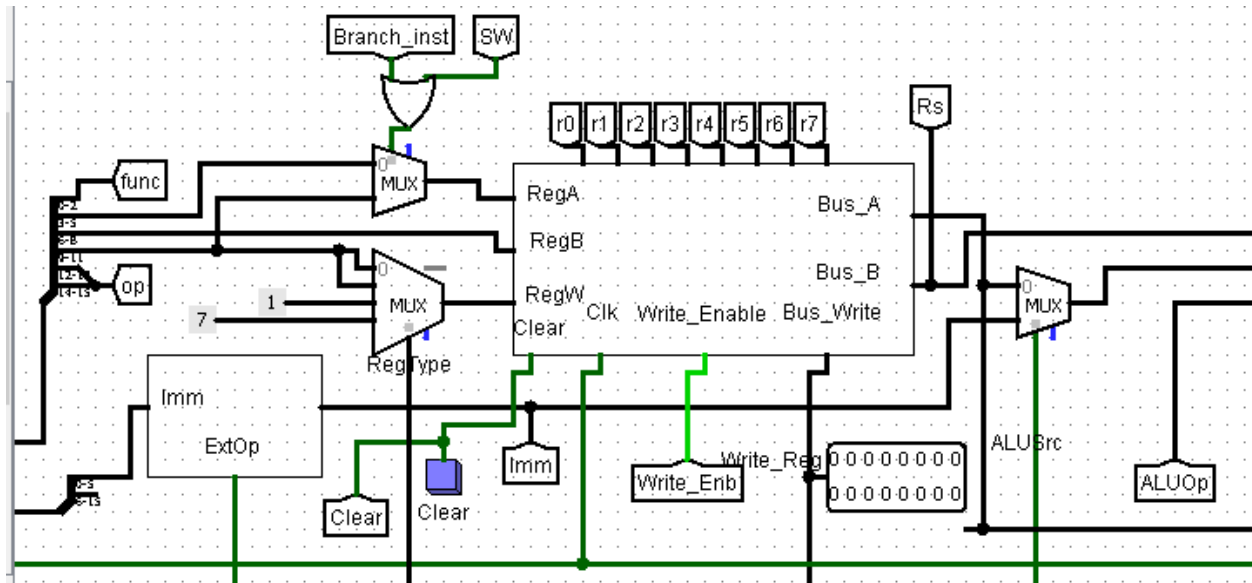
Datapath

1) Instruction Fetch:



The Instruction Fetch (IF) stage consists of a PC register and an Instruction Memory that reads the instructions from the memory based on the PC. PC_Sel selects between PC + 1, PC + 1 + Offset for branch instructions, Immediate for J and JAL instructions, and Rs value for JR and JALR instructions.

2. Instruction Decode:

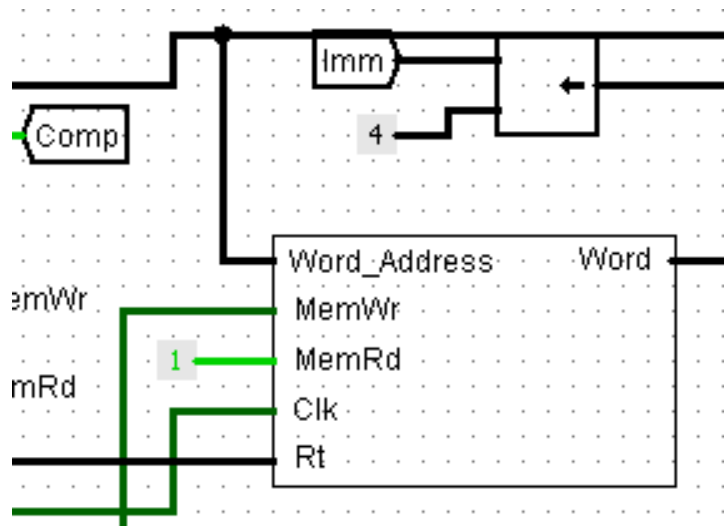


During this stage, the processor decodes the instruction and reads the register file and generates the main control signals (except branch_taken and mov_taken which will be found in the execute cycle). The input of RegA in the register file is bits 3 – 5 any time unless the instruction was a branch or an SW, where the input is bits 9 -11 at this point. The input for RegB is always bits 6 – 8. The input for RegW is bits 9 – 11 usually, or R1 during LUI or R7 during JAL. At the output, BusA is multiplexed with the extended immediate number to be an input for the ALU.

Specific details of the input of blocks and control signals will be discussed in their respective parts.

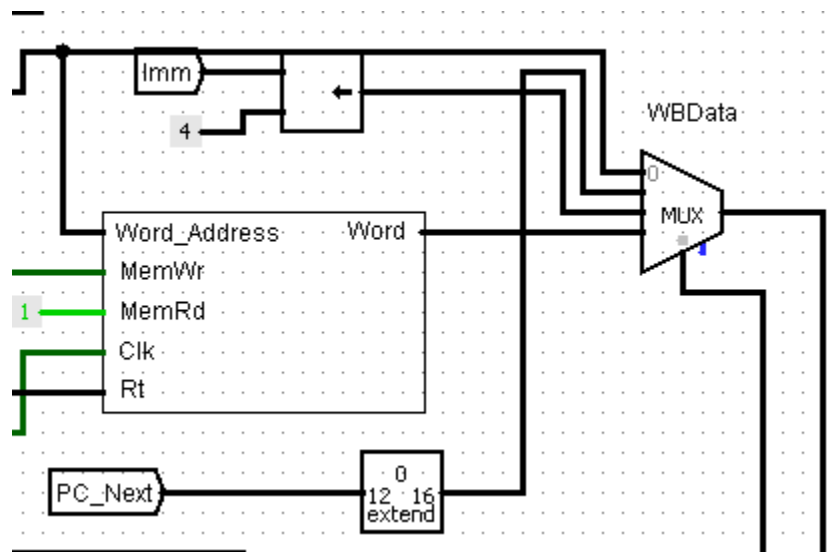
Keep in mind that the ALU does not produce a zero bit flag, rather it decides whether the branch instruction is satisfied (Comp == 1) or not (Comp == 0).

4. Data Memory Stage



During the data memory stage, data is either stored in the memory (SW operation), with $Rt == \text{BusA}$ from the RegFile, or data is loaded into one of the registers in the register file (Load operation). In both cases, the address is computed as an offset from a base address in the ALU and is fed as **Word_Address** in the memory. Keep in mind that **MemRd** is always 1 in our design, as we actually do not care if the memory is read or not, as long as we do not store the read values into the register except during Load operation. **MemWr**, however, is only 1 during SW operation.

5. Write Back Stage – WB Stage



During this stage, different outputs are fed back to the register file depending on the instruction.

$\text{Imm} \gg 4$ shifted to the left is stored in the RegFile during LUI operations.

PC_Next with zero extension is stored in the RegFile during JAL and JALR operations.

Memory output is stored in the RegFile during Load operations.

Else, the ALU output is stored in the RegFile.

Keep in mind that the Write_Enable signal is not ON all the time, meaning that during some instructions, we will not store anything in the RegFile regardless of the MUX output. This will be discussed in the control signals section.

- **Main Control Signals:**

The main control signals are the main signals that are generated to control the datapath and the select lines of the MUXes in the CPU.

The signals were divided into 4 parts:

Decode Stage Signals: Consists of most main control signals

ALU Control Signals: Consists of signals that control ALU execution

Reg_Enb Signal: For writing to the RegFile (1), else (0)

PC Control Signals: Branch instructions, jump instructions, branch taken or not taken

Control Signals Table

$+$ = **OR** | x = **AND** | $'$ = **NOT** | **DNC** = **DO NOT CARE** | **X** = **DO NOT CARE (1 – BIT)**

Opcode	Func	Signal Output	Value
1XXX + X1XX	DNC	ExtOp	1 – Else 0
0101 (SW)	DNC	Mem_Write	1 – Else 0
0110 (Load)	DNC	WBData	11
1101 (LUI)	DNC	WBData	10
0001	111 (JALR)	WBData	01
1111 (JAL)	DNC	WBData	01 – Else 00
0111 + 1000 + 1001 + 1010 + 1011 + 1100	DNC	Branch_Inst	1 – Else 0
111X	DNC	J_Inst	1 – Else 0
0001	11X	JR_Inst	1 – Else 0
JR_Inst	JR_Inst	PC_Sel	11
J_Inst	J_Inst	PC_Sel	10
Branch_Inst	Branch_Inst	PC_Sel	01 – Else 00
Branch_Inst + Comp_Out	Branch_Inst + Comp_Out	Branch_Taken	1 – Else 0
0001	10X	Mov_Inst	1 – Else 0
Mov_Inst x Mov_Sat	Mov_Inst x Mov_Sat	Mov_Taken	1 – Else 0
{Opcode, Func}	{Opcode, Func}	ALUOp	{Opcode, Func}
111X + Branch_Inst'	111X + Branch_Inst'	ALUSrc	1 – Else 0
11X1	DNC	Reg_Sel	11
111X	DNC	Reg_Sel	10
000X	DNC	Reg_Sel	01 – Else 00
(JR_Inst' x Mem_Write' x Mov_Inst') + Mov_Taken	(JR_Inst' x Mem_Write' x Mov_Inst') + Mov_Taken	Reg_Write	1 – Else 0
0000	000	Logic_Sel	00
0000	001	Logic_Sel	01

0000	010	Logic_Sel	10 – Else 11
DNC	1XX	Shift_Mov_Op	1 – Else 0
0000	101	Sub_Op	1 – Else 0
Sub_Op + Branch_Inst	Sub_Op + Branch_Inst	Add_Sub_Sel	1 – Else 0
Shift_Mov_Op	Shift_Mov_Op	ALU_Sel	00
Add_Op + Sub_Op	Add_Op + Sub_Op	ALU_Sel	01
0000	000 + 001 + 010 + 011	ALU_Sel	11 – Else 10

1. Decode Stage Signals

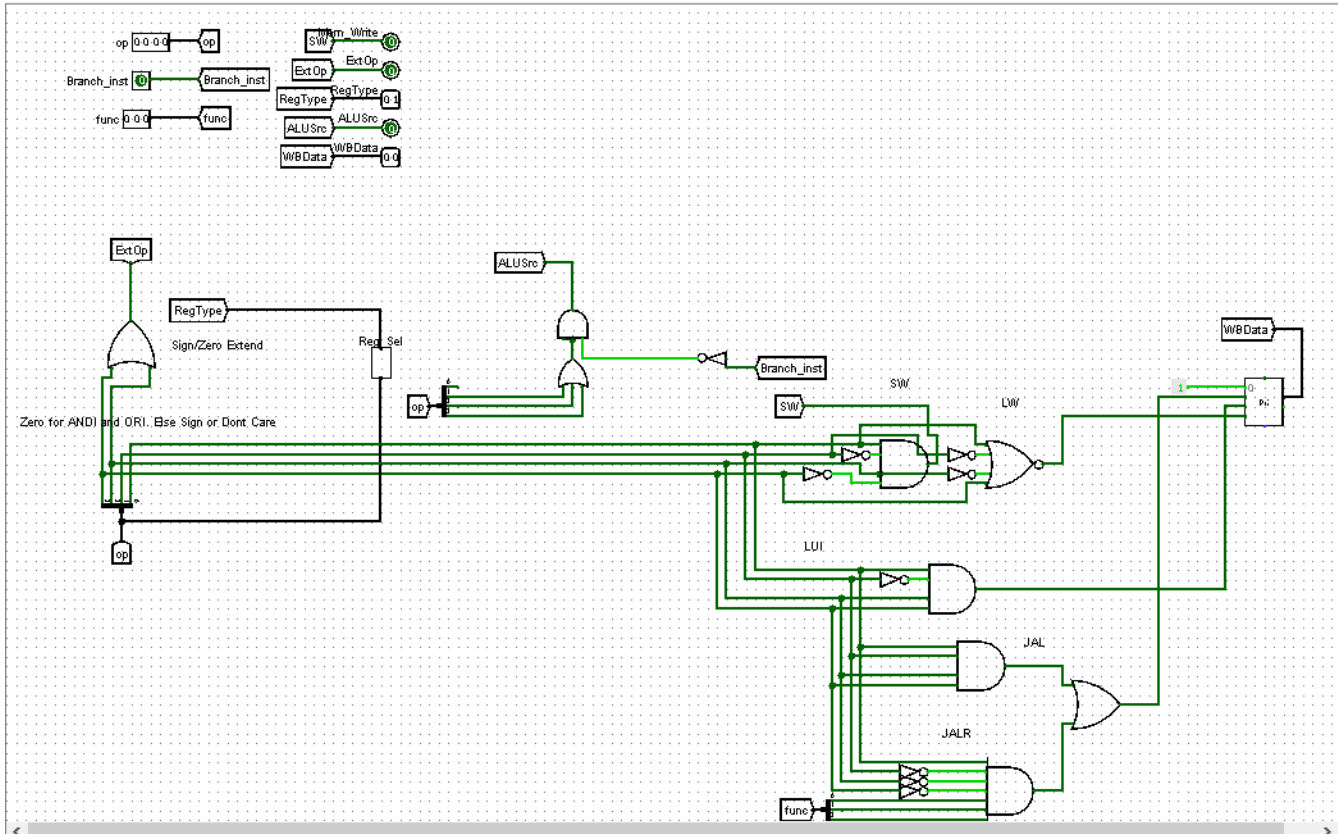
ExtOp : Determines whether the immediate should be zero extended or sign extended

ALUSrc: Chooses between immediate value or BusA

Mem_Write: Write to the memory (SW) or not

WBData: Chooses input to write back (ALU Output - 00, Imm >> 4 - 01, {PC_Next,0000} - 10, Data Memory Output 11)

RegType: (Bits 6 – 8 – 00 or 01, Reg1 – 10, Reg7 – 11)



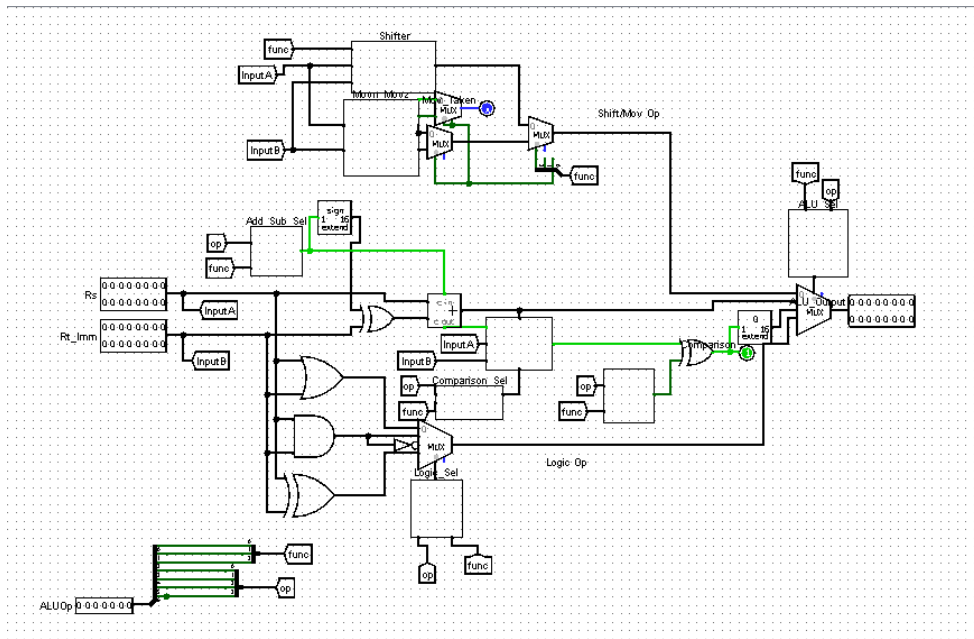
2. ALU Control Signals

Logic_Sel: Selects between what logic operation to do (00 – OR , 01 – AND, 10 – NAND, 11 – XOR)

Add_Sub_Sel: Selects between doing addition (0) or subtraction (1) (in 2's complements)

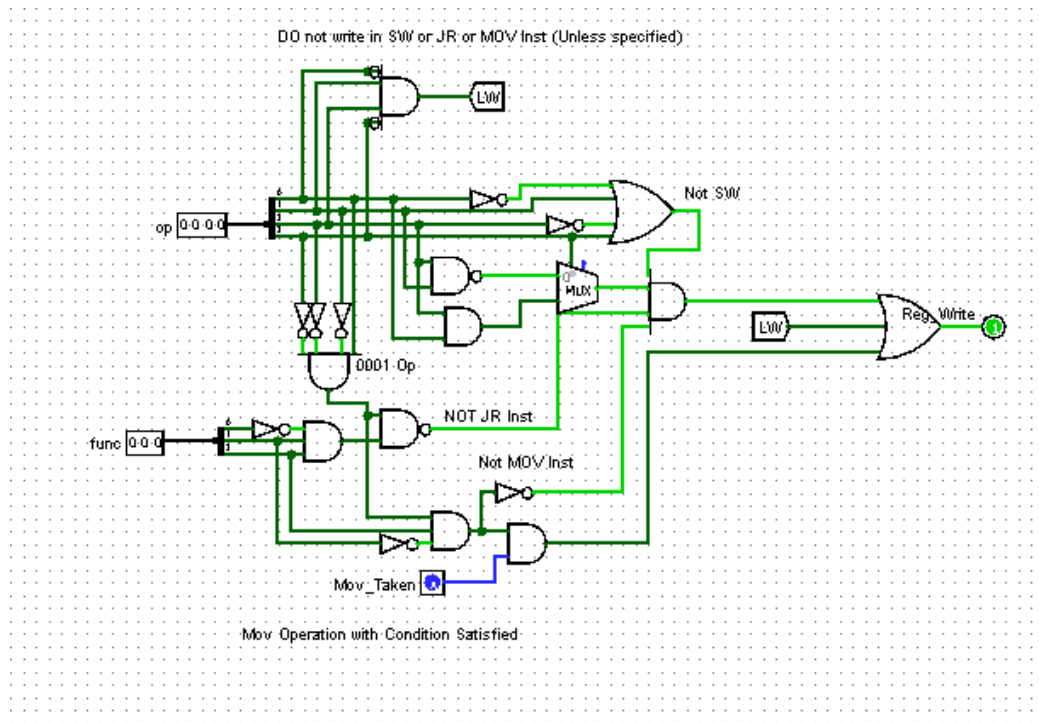
Shift_Mov_Op: Selects between shift operation (0) or MOVZ/MOVN (1)

ALU_Sel: Selects the final output from Shift_Mov (00), Add_Sub (01), Comparison (10), Logic (11)



3. Reg_Write Signal

0 – DO NOT write to RegFile, 1 – WRITE to RegFile



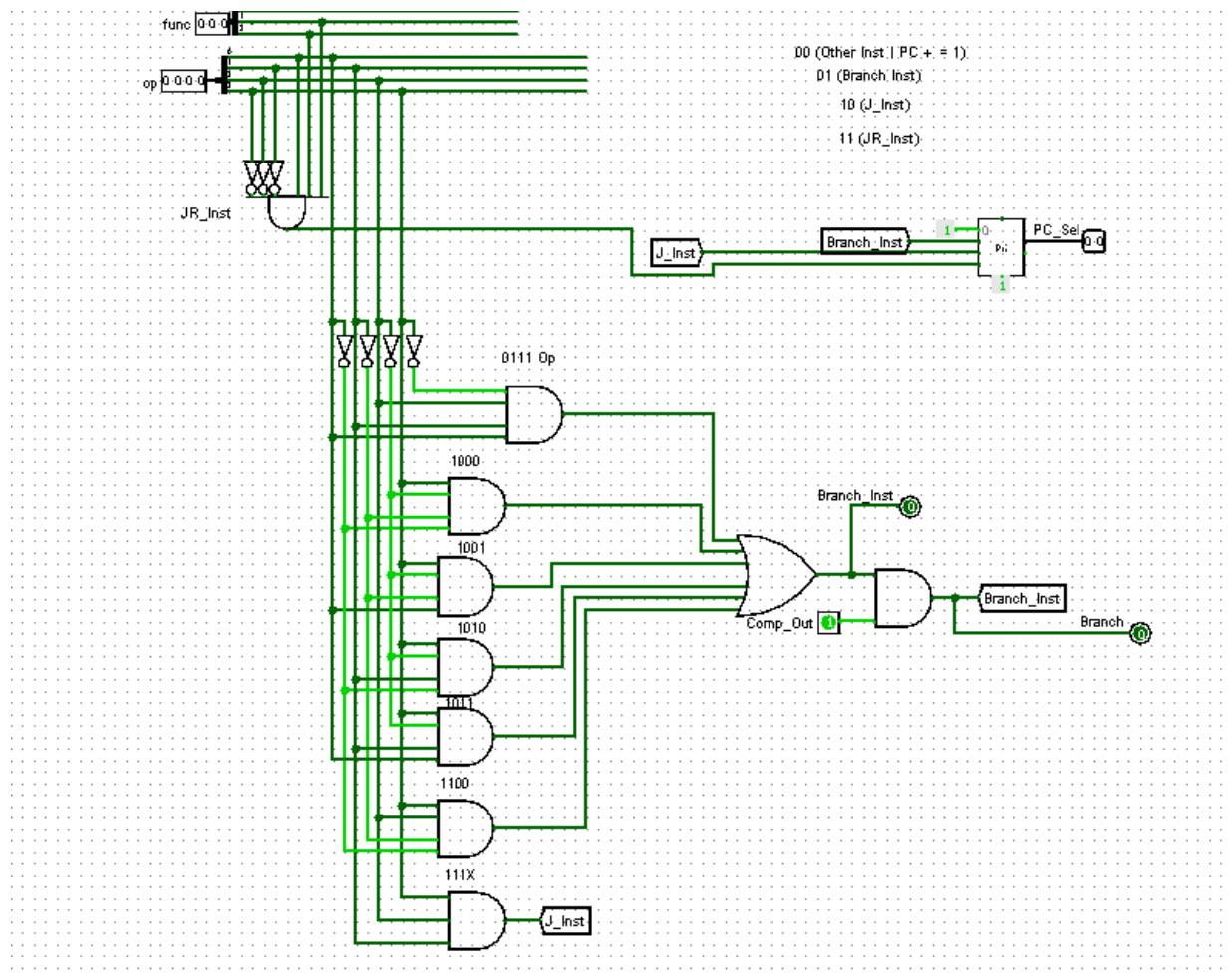
4. PC_Control_Signals

Branch_Inst : If the instruction is branch (1) or not (0)

Branch_Taken: If the branch is taken (1) or not (0), it is by default 0 for non – branch instructions

PC_Sel: Selects between PC_Next (00), PC_Next + Offset (01), Imm[11:0] (10), Rs Register (11)

JR_Inst → 11 Select, J_Inst → 10, Branch_Inst → 01, Else 00



Pipelined Processor

The pipelined processor aims to maximize the utilization of the CPU components by keeping the 5 parts of the CPU busy with different instructions at the same time. However, this often causes hazards as many stages depend on each other. Therefore, pipelining should be able to handle these type of situations.

Procedures for pipelining

- 1) Pipeline Registers: These registers prevent the information of different instructions to spoil the information of the current instruction at a certain stage by using sequential elements. Therefore, different stages of the CPU depend on information from the pipelined registers passed to it from the stage before it. Pipeline registers include both data registers and control registers
- 2) Hazards and Dependencies: Hazards and dependencies include any interference from other stages or dependency of instructions on each other that may interfere with the instructions in the CPU.
 - Structural Hazards:
Occurs when two different instructions require access to the same hardware at the same time. E.g. two instructions accessing the RegFile
Solution: Delay one instruction until the other finishes that stage through pipeline registers
 - Data Hazards:
 - a) Data dependency: When the input of one instruction depends on the output of another instruction, but the other instruction is still not finished through the WB stage
Solution: Use forwarding to bypass the stages. Use MUXs to take the output from ALU stage, Data Memory Stage, or WB stage instead of waiting it until it returns to the CPU.
 - b) Load Stalls: If there is a data dependency from a load instruction, then the output can not be bypassed from the ALU stage as the output is still not fetched from the data memory. Therefore, there needs to be a stall of one cycle until the data is fetched from the memory, so that the previous instructions can proceed.
 - Control Hazards: Branch and jump instructions means the CPU has to stop all previous instructions from branch and jump from working in the pipeline. A branch statement output can only be found in the ALU cycle, which means we MIGHT have to stop two instructions in the ID and IF cycle depending on the branch output (taken or not) and our prediction by killing the two instructions. However, for jump instructions, the instruction is decoded during the ID stage, which means only one instruction (in the IF stage) needs to be killed.

Our pipelined processor follows the mentioned steps and solutions, as there exists other solutions, but we decided to follow with these mentioned steps.

During the pipelining, signals are propagated forwards throughout the pipeline in order to reach the next stage of the pipeline. Typically, they are just propagated via registers until they are needed. The write_enable signal, however, is ANDed with Mov_Taken flag during the execute stage. This is done since if the mov condition is not satisfied, we cannot write back.

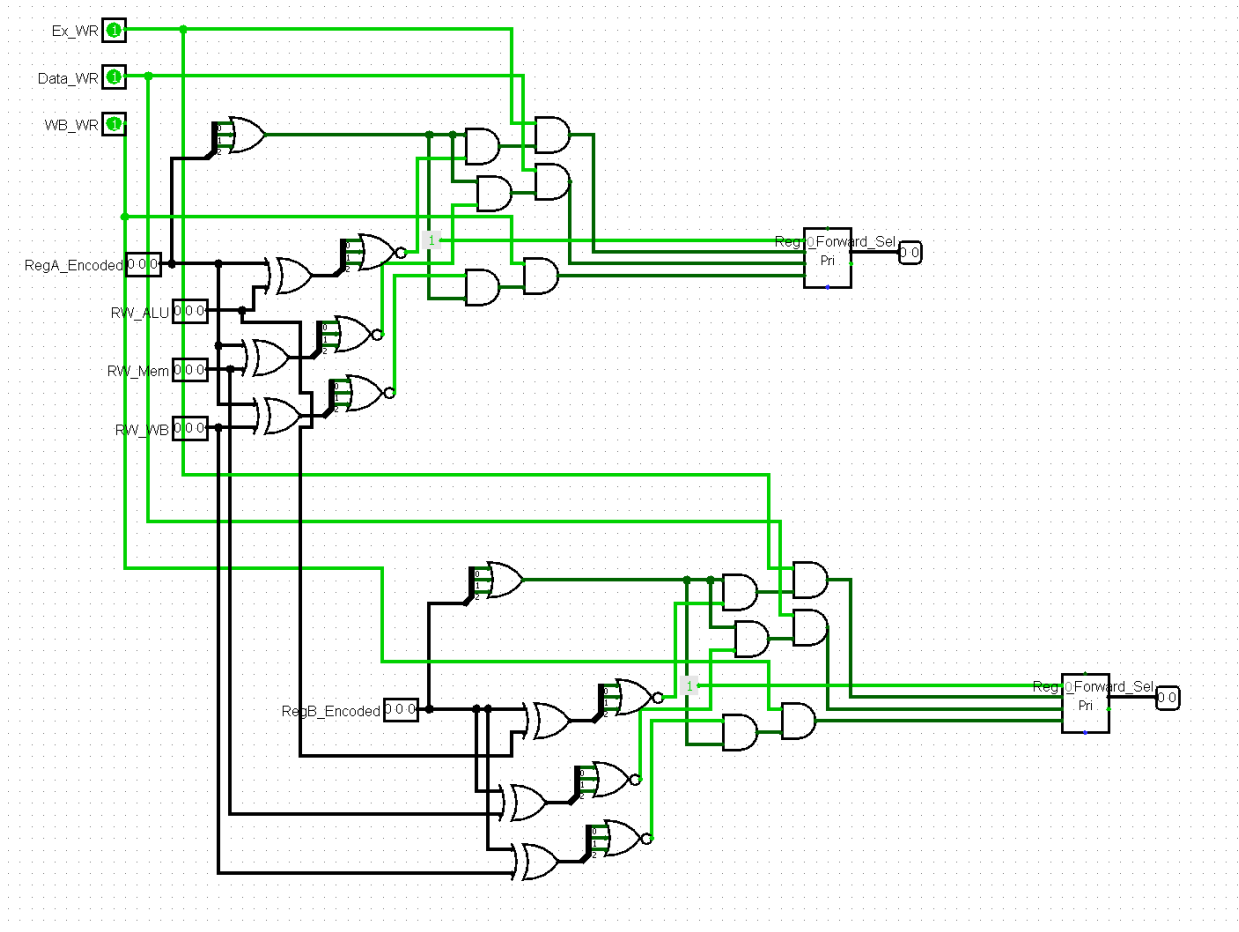
Data Hazards

Forwarding

In our pipelined processor, we deal with data dependencies through forwarding. This is done through multiplexers that take forward signals from different stages in the pipeline to and chooses them accordingly via a forward control. These are the signals and their origin.

- Forward 1
 - From the execute stage
- Forward 2
 - From the memory stage
- Forward 3
 - From the writeback stage

They are each passed into a multiplexer along with the typical values that result during the decode stage (Rs and Rt). This creates two multiplexers with inputs Forward 1-3 and the respective register. The select line for the multiplexer is controlled by the forward control which has the following structure:



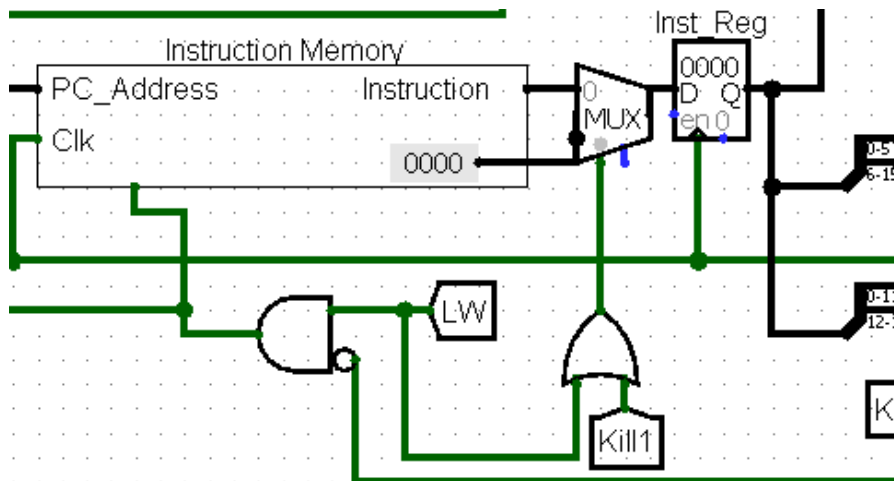
It uses values from the writeback registers in the pipeline to determine if forwarding is needed and which forward to use. It follows this logic:

If $((Rs \neq 0) \text{ and } (Rs == Rd2) \text{ and } (EX.RegWr))$	ForwardA = 1
Else if $((Rs \neq 0) \text{ and } (Rs == Rd3) \text{ and } (MEM.RegWr))$	ForwardA = 2
Else if $((Rs \neq 0) \text{ and } (Rs == Rd4) \text{ and } (WB.RegWr))$	ForwardA = 3
Else	ForwardA = 0

If $((Rt \neq 0) \text{ and } (Rt == Rd2) \text{ and } (EX.RegWr))$	ForwardB = 1
Else if $((Rt \neq 0) \text{ and } (Rt == Rd3) \text{ and } (MEM.RegWr))$	ForwardB = 2
Else if $((Rt \neq 0) \text{ and } (Rt == Rd4) \text{ and } (WB.RegWr))$	ForwardB = 3
Else	ForwardB = 0

Stalling

Although data dependencies can be resolved with forwarding, this isn't true for all data dependencies. Dependencies that require memory loading will instead rely on stalling. Stalling comes after a LW instruction where the next instruction is paused for 1 cycle. This is achieved by killing the current instruction and disabling the PC register (set the enable to 0). This would only occur during the decode stage of the LW. Whether an instruction is a LW is calculated from the main control during the decode stage. To kill the current instruction from being fetched, a multiplexer is added after the instruction memory. LW will be the select. In the case it is not a LW, the instruction will fetch normally, otherwise it will fetch 0x0000 instruction. This can be seen here:

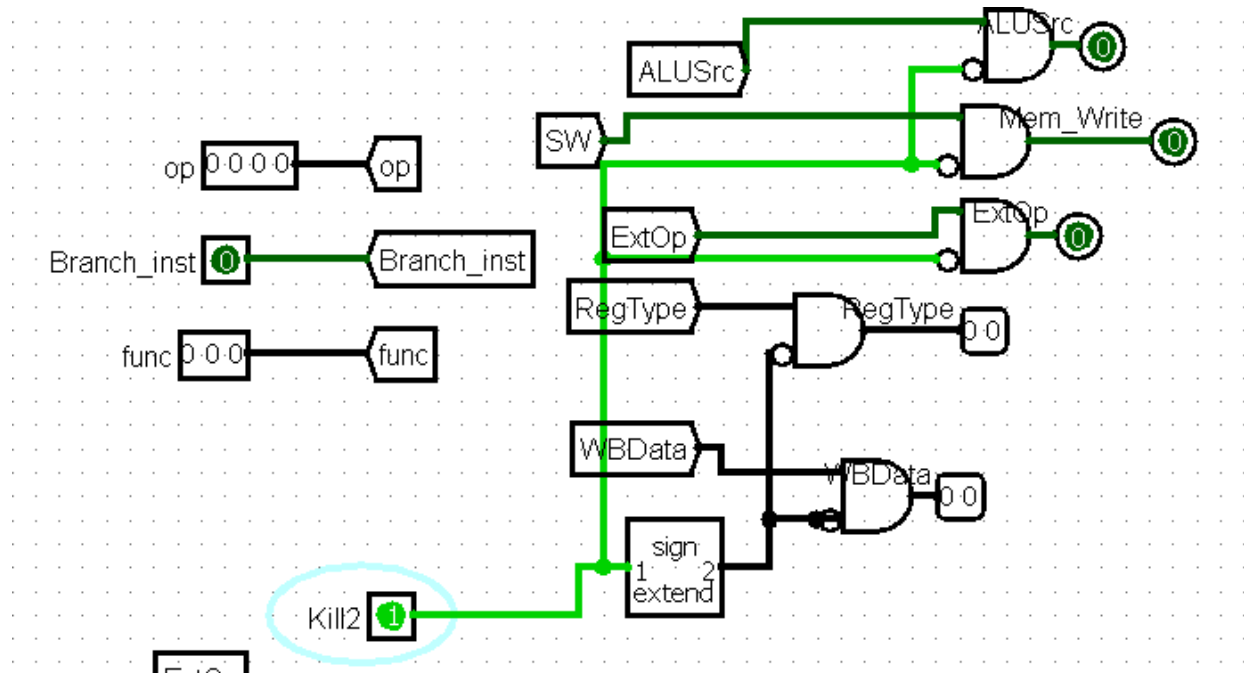


LW is also ANDed with Kill2 complement in the PC enable and ORed with Kill1 for the select line for control hazards explained later on.

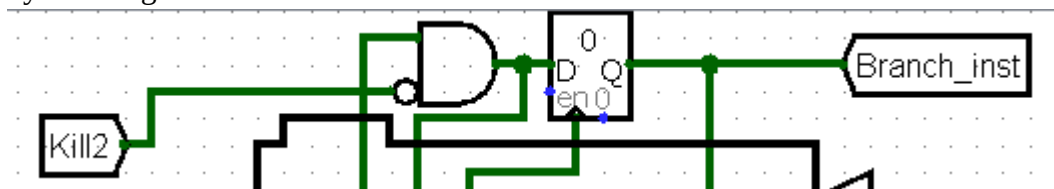
Control Hazards

When a branching instruction is executed, the processor continues as usual by fetching the next instruction. This is because the processor is not sure if the branching will occur or not. To maximize efficiency, it will continue normally and if it turns out that the branching is occurring, then the fetched instructions will be killed and the branching will occur. The killing happens through the signals Kill1 and Kill2. Kill1 is used to kill any new instructions that are being fetched. As such, it can be ORed with LW as an input to MUX. Kill2 is used to kill instructions that have reached the decode stage. This can be achieved by passing it as an input to the main control which would disable all other control signals as can be seen below.

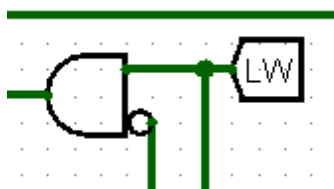
Jump instructions only require Kill1 as they do not need to wait until the execute stage to find out if the jump is happening. Therefore, the next instruction will only reach the fetch stage before it is killed making a need for Kill2 for it unnecessary.



Kill2 is also used to resolve certain cases. For instance, if a branch instruction or a LW instruction appears after the branch, this can lead to issues as LW and branch instructions may have effects during the decode stage itself where LW may cause stalling and branch instructions may influence the execute stage due to the branch_instruction flag. Kill2 can handle these cases by ANDing with them.



Stalling signal as the ANDing of LW and Kill2 compliment



2) Simulation and testing

For simulating and testing, crucial components were tested by running every possible instruction and checking for edge cases. This can be seen with the ALU where all instructions were tested, and results were compared to ideal results. The main program used for testing was the bubble sort algorithm as it contains many instructions and combinations of instructions. It has branching memory loading and storing, ALU instructions, and immediate instructions. The program also includes combinations that promote stalling, killing, and all types of forwarding. This made it perfect for testing our CPU. The assembly code for the algorithm can be seen below. Unfortunately, this program does not test all instructions individually but that was resolved through manual testing.

No known bugs were found, and all instructions worked appropriately. All instructions were implemented.

Pipelining Hazards:

- Data Hazards
 - Forward 1:
 - Handles forwarding from the execute stage
 - Forward 2:
 - Handles forwarding from the memory stage
 - Forward 3:
 - Handles forwarding from the writeback stage
 - LW:
 - Stalling was used during LW instructions
- Control Hazards:
 - Killing:
 - Killing was used to kill any instruction executed in the case of branching
 - Kill was used along the pipeline
 - Kill1 was used to kill any new instruction
 - Kill2 was used to kill already fetched instructions
 - LW stalling was killed through Kill2 if LW came after a branch
 - Branching instructions were killed through Kill2 if they came after an executed branch

Bubble Sort

```
1  # 1 n-1
2  # 2 i, j
3  # 3 swapped
4  # 4 n-1-i
5  # 5 6 arrj arrj+1
6  addi $1, $0, 7
7  for1:
8  xor $3, $3, $3
9  sub $4, $1, $2
10 sw $2, $0, 16
11 xor $2, $2, $2
12 for2:
13 lw $5, $2, 0
14 lw $6, $2, 1
15 ble $6, $5, 6 #for2_cont
16 add $7, $5, $0
17 add $5, $6, $0
18 add $6, $7, $0
19 sw $5, $2, 0
20 sw $6, $2, 1
21 addi $3, $0, 1
22 for2_cont:
23 addi $2, $2, 1
24 blt $4, $2, -11 #for2
25
26 beq $3, $0, 3 #end
27 lw $2, $0, 16
28 addi $2, $2, 1
29 blt $1, $2, -19 #for1
30 end:
31
32 # void bubbleSort(int arr[], int n)
33 #{
34 #     int i, j;
35 #     bool swapped;
36 #     for (i = 0; i < n - 1; i++) {
37 #         swapped = false;
38 #         for (j = 0; j < n - i - 1; j++) {
39 #             if (arr[j] > arr[j + 1]) {
40 #                 swap(&arr[j], &arr[j + 1]);
41 #                 swapped = true;
42 #             }
43 #         }
44 #
45 #         // If no two elements were swapped by inner loop,
46 #         // then break
47 #         if (swapped == false)
48 #             break;
49 #     }
50 #}
51
```

Machine code

4207 06db 0855 5410 0493 6a80 6c81 cd46

0f44 0b84 0dc4 5a80 5c81 4601 4481 b8b5

8603 6410 4481 b2ad

Bubble Sort Testing

Before:

Instruction Memory:

```
000 4207 06db 0855 5410 0493 6a80 6c81 cd46 0f44 0b84 0dc4 5a80 5c81 4601 4481 b8b5
010 8603 6410 4481 b2ad 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
```

Data Memory:

```
-----
000 0023 0012 00a5 005c 001e 0083 0003 0124
-----
```

After:

Data Memory:

```
-----
000 0003 0012 001e 0023 005c 0083 00a5 0124
-----
```

Counter Program (counting the number of 1s in a register)

```

1  # Checks for 1s in $7
2  # $2 has the result
3  addi $7, $0, 0b011010
4  addi $1, $0, 1
5  addi $4, $0, 1
6  while:
7  and $3, $1, $7
8  add $2, $2, $3
9  srl $7, $7, $4
10 beq $7, $0, 1 # end
11 j 5 # while
12 end:

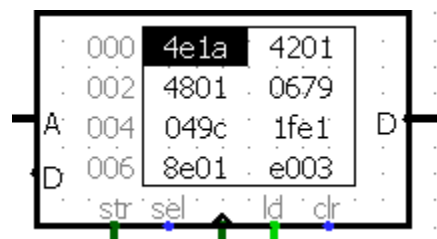
```

Machine code

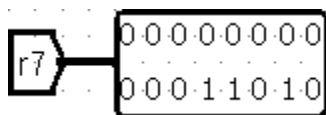
4e1a 4201 4801 06790 49c 1fe1 8e01 e003

Testing

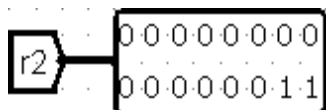
Instruction Memory:



Register to count:

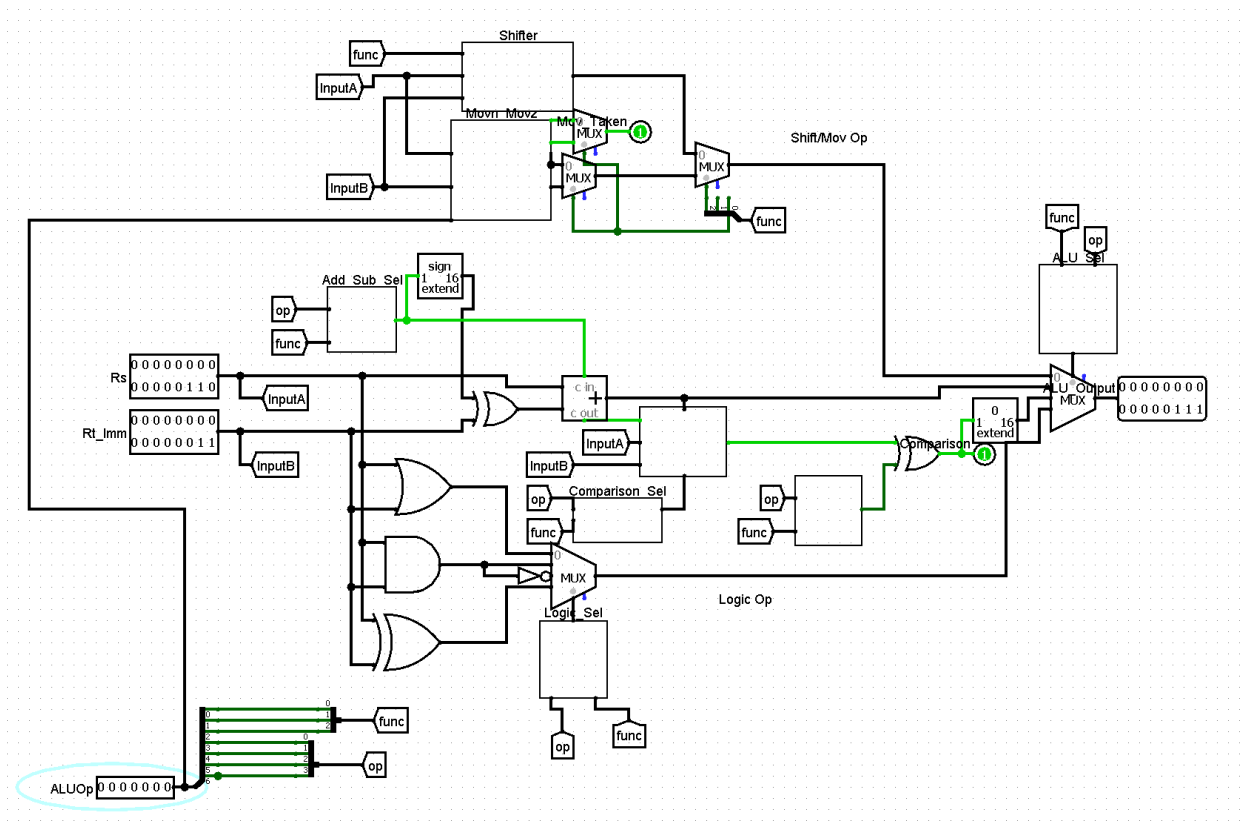


Count after execution:

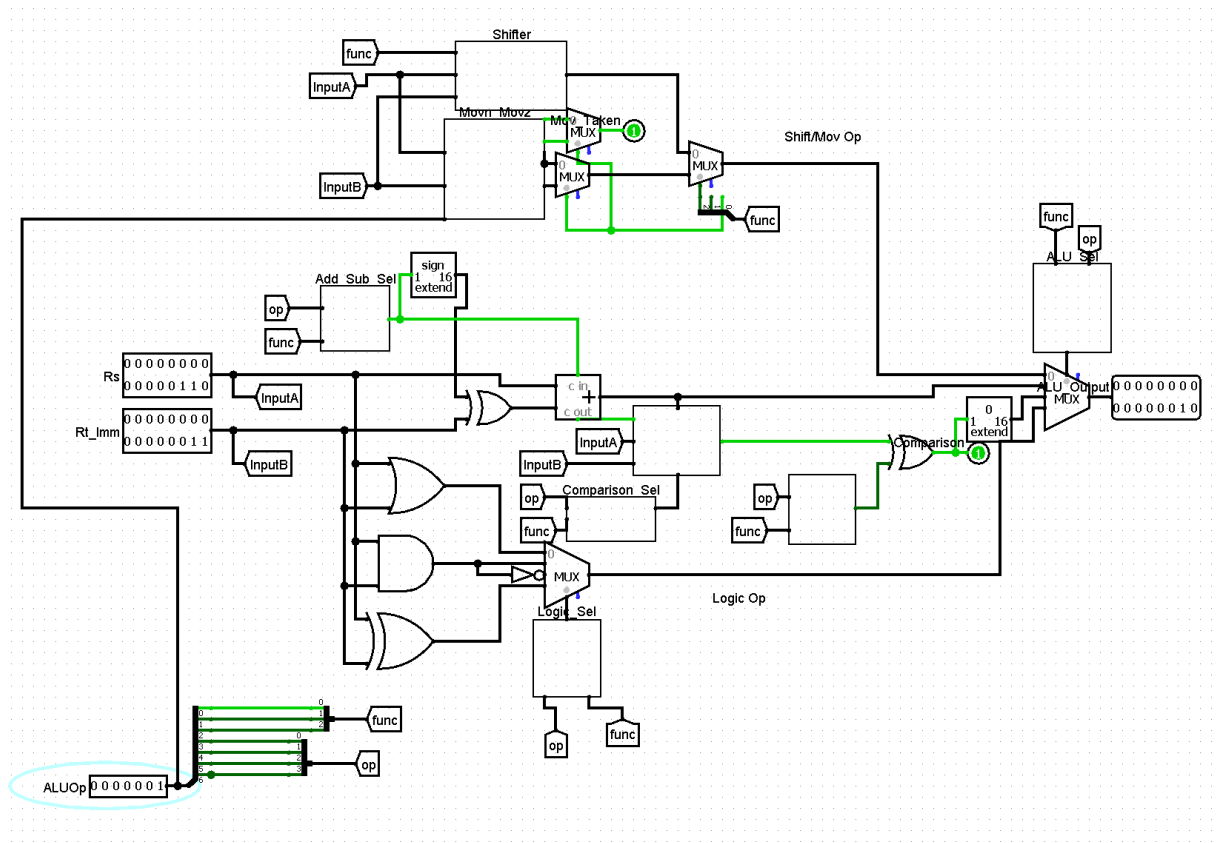


ALU

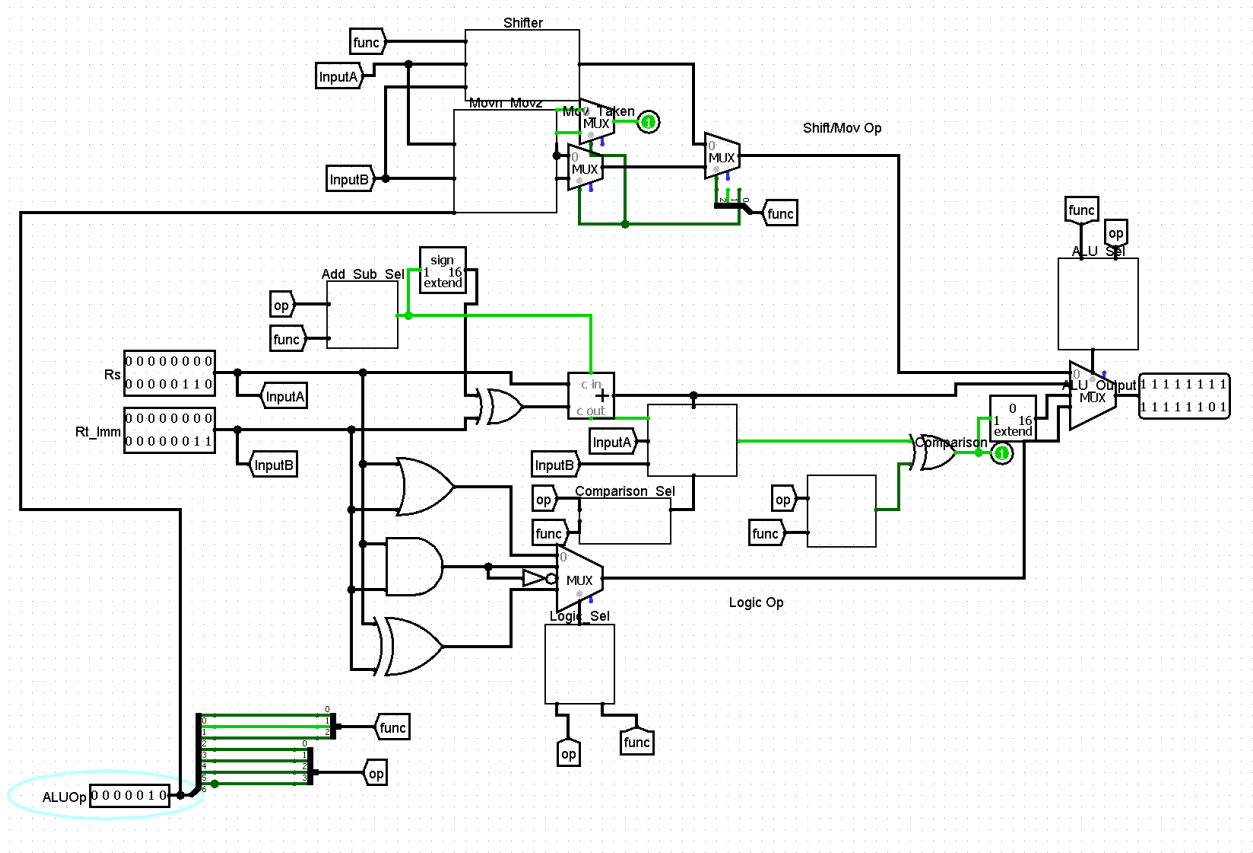
OR:



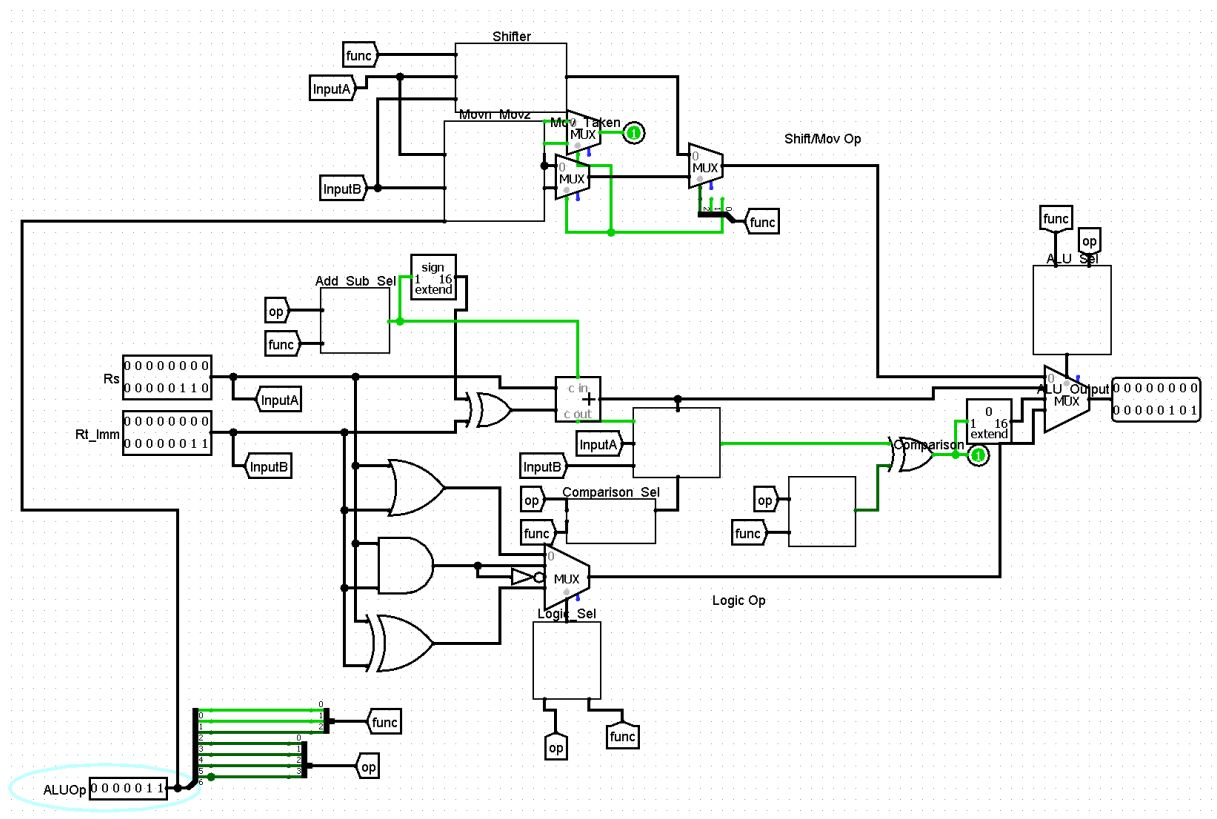
AND:



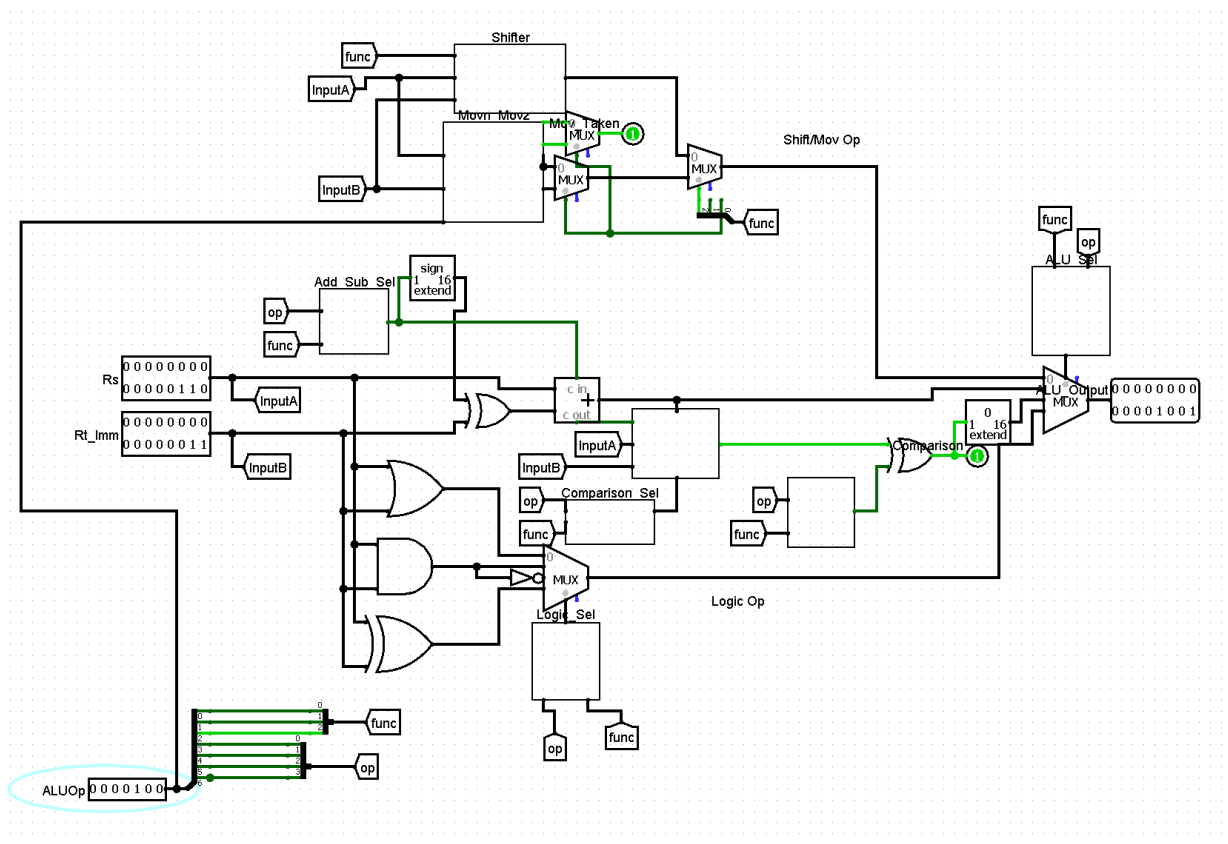
NAND:



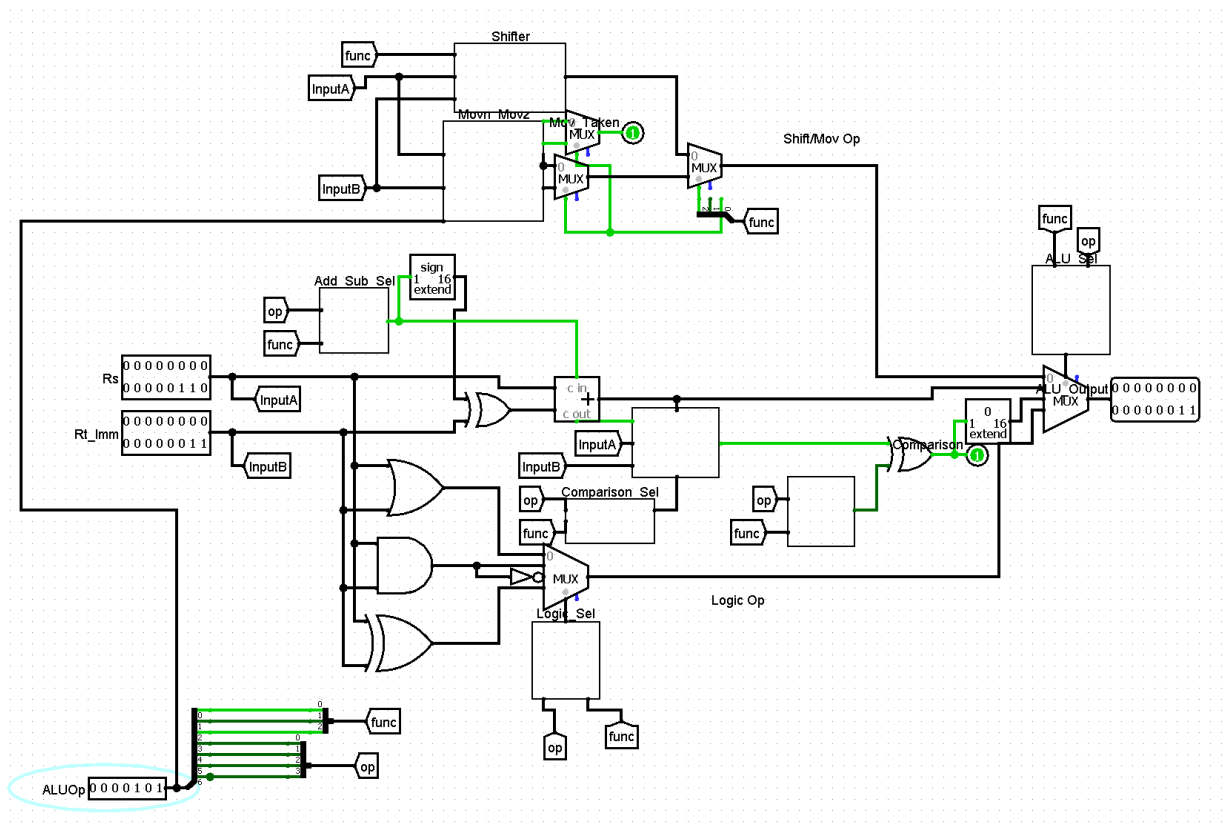
XOR:



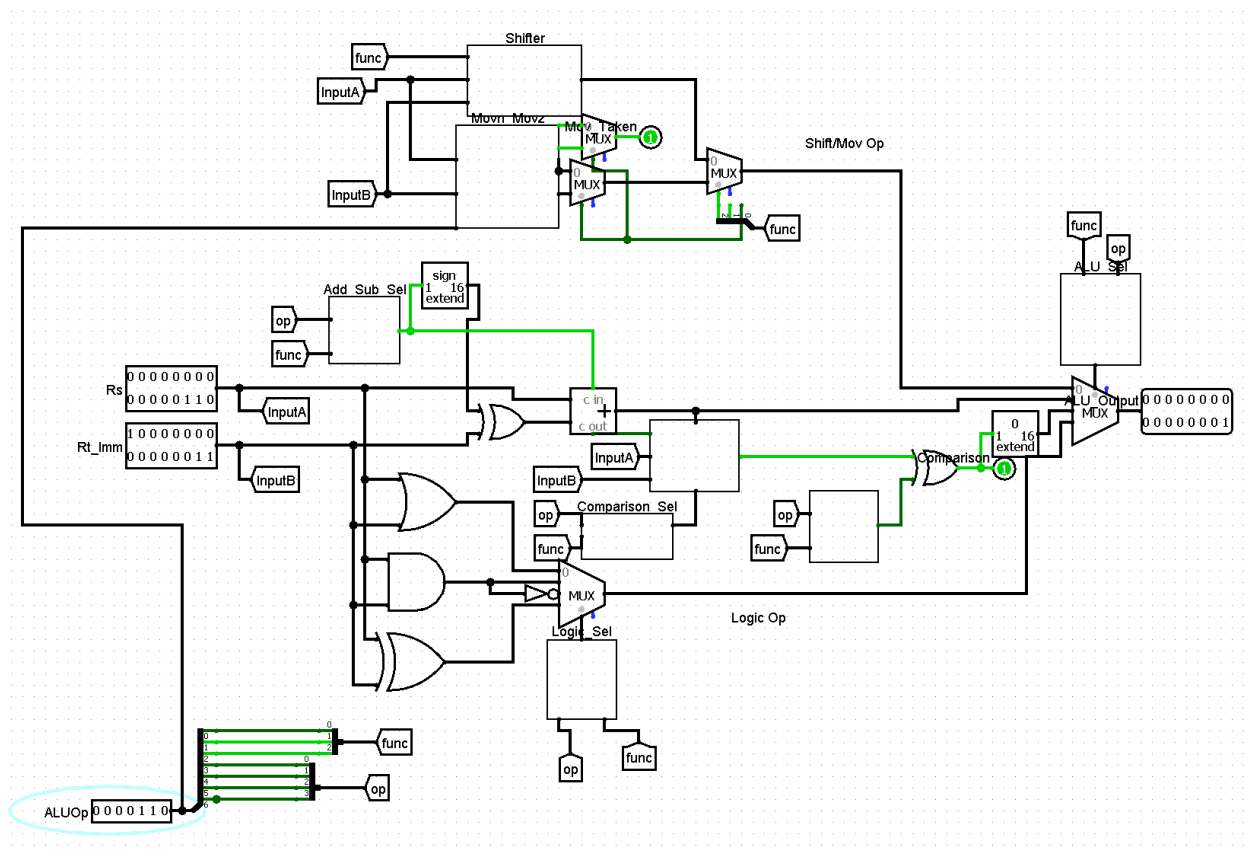
ADD:



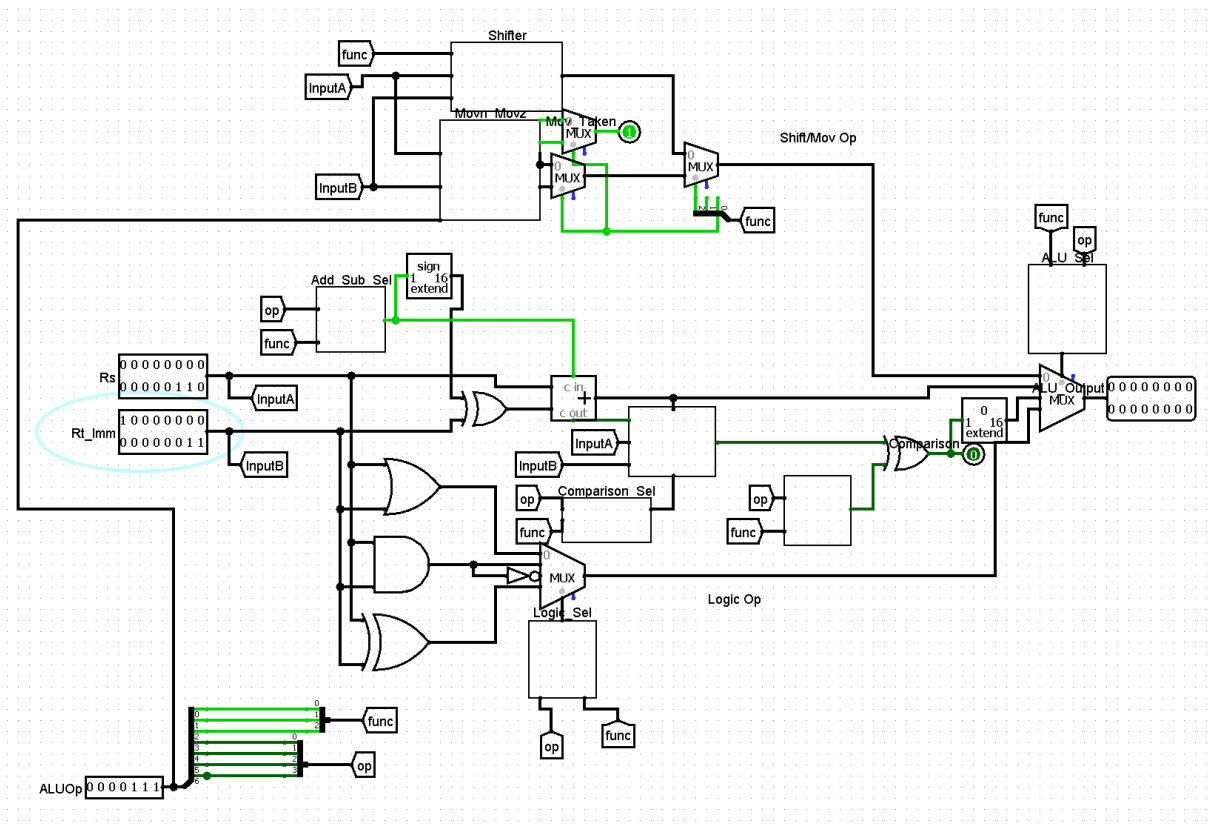
SUB:



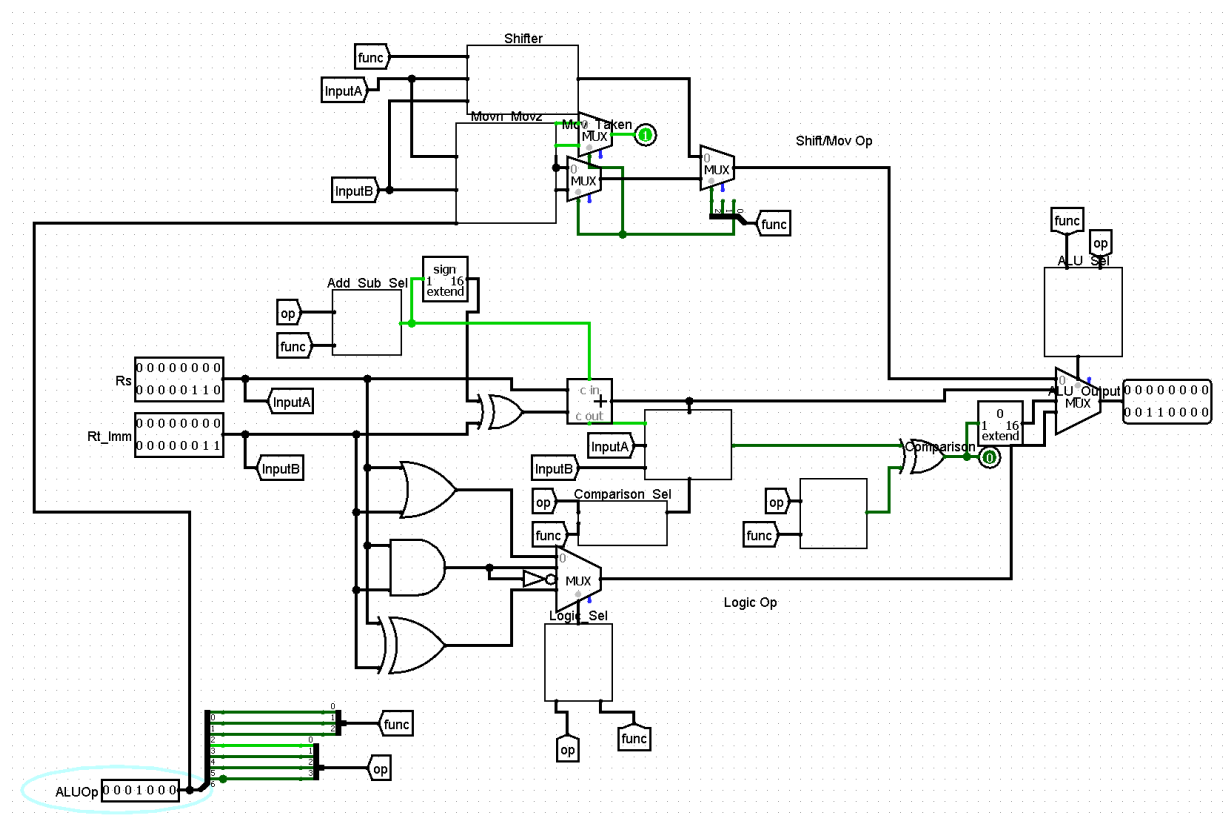
SGE:



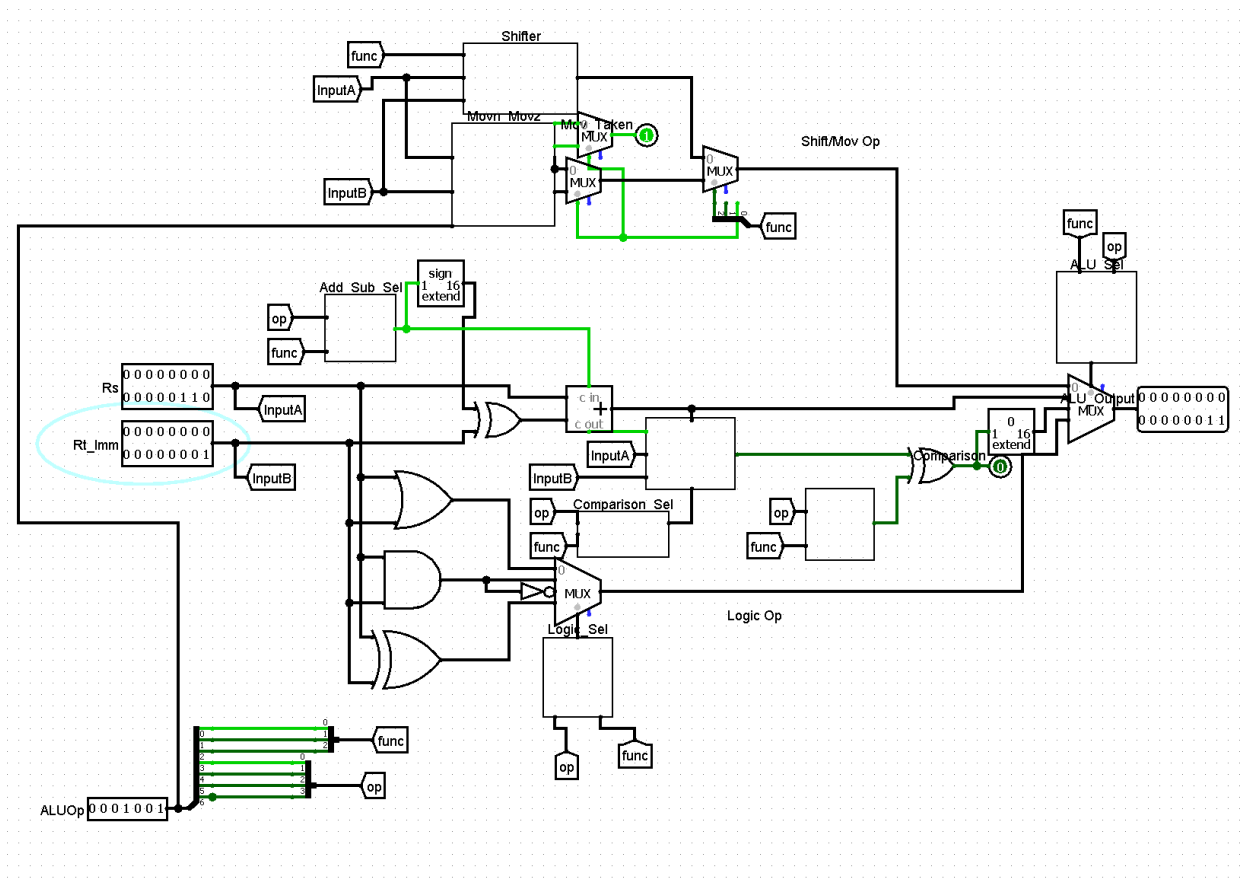
SGEU:



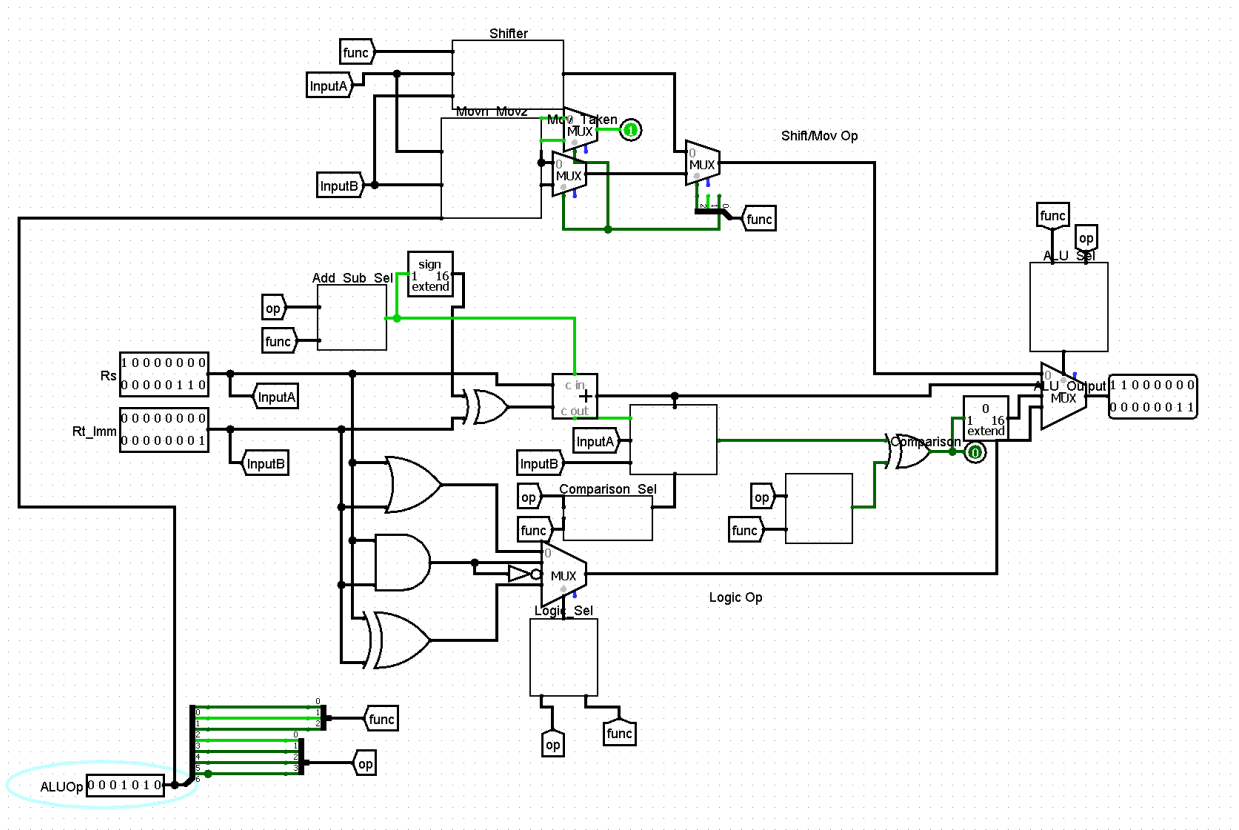
SLL:



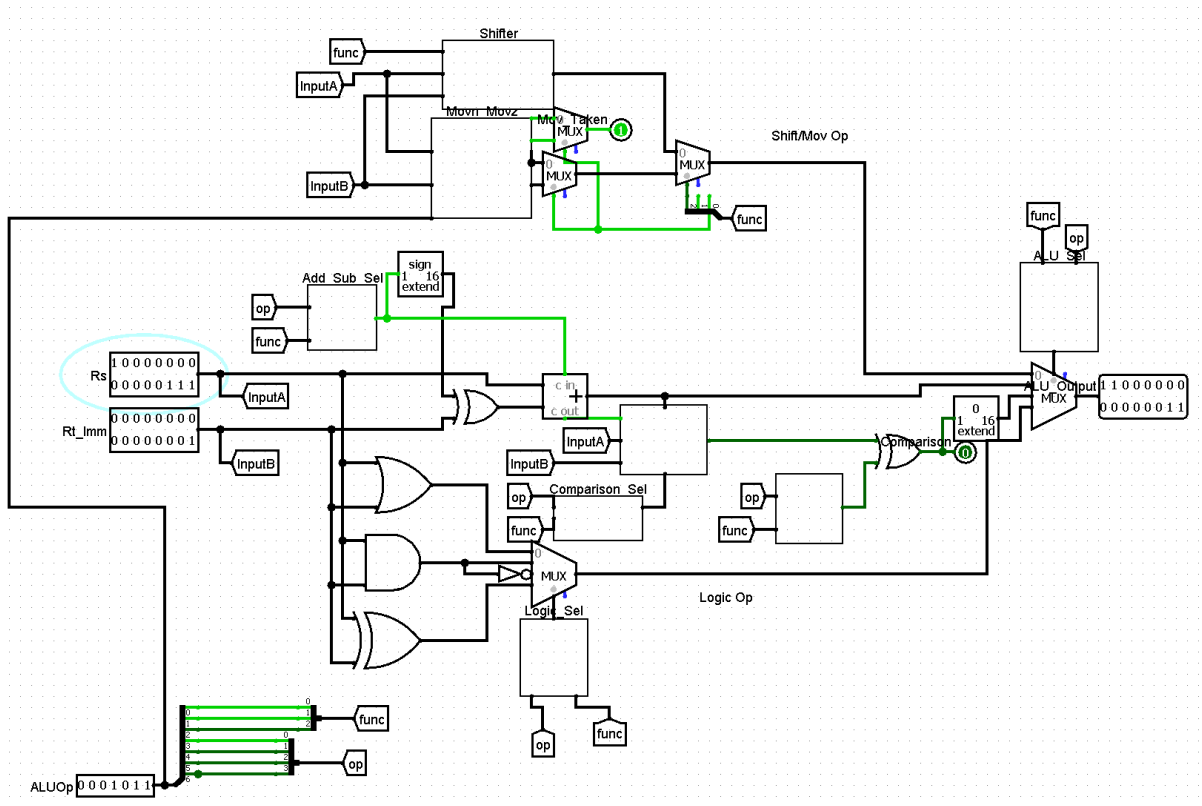
SRL:



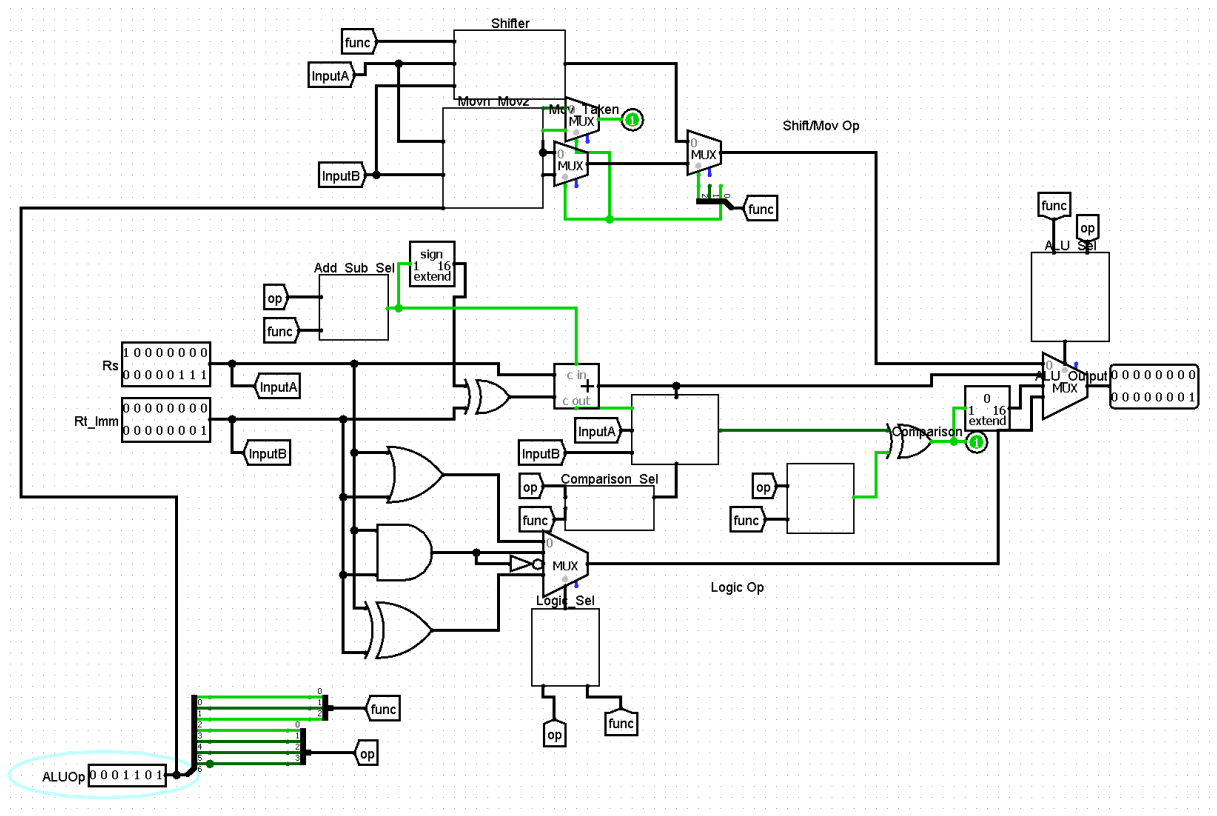
SRA:



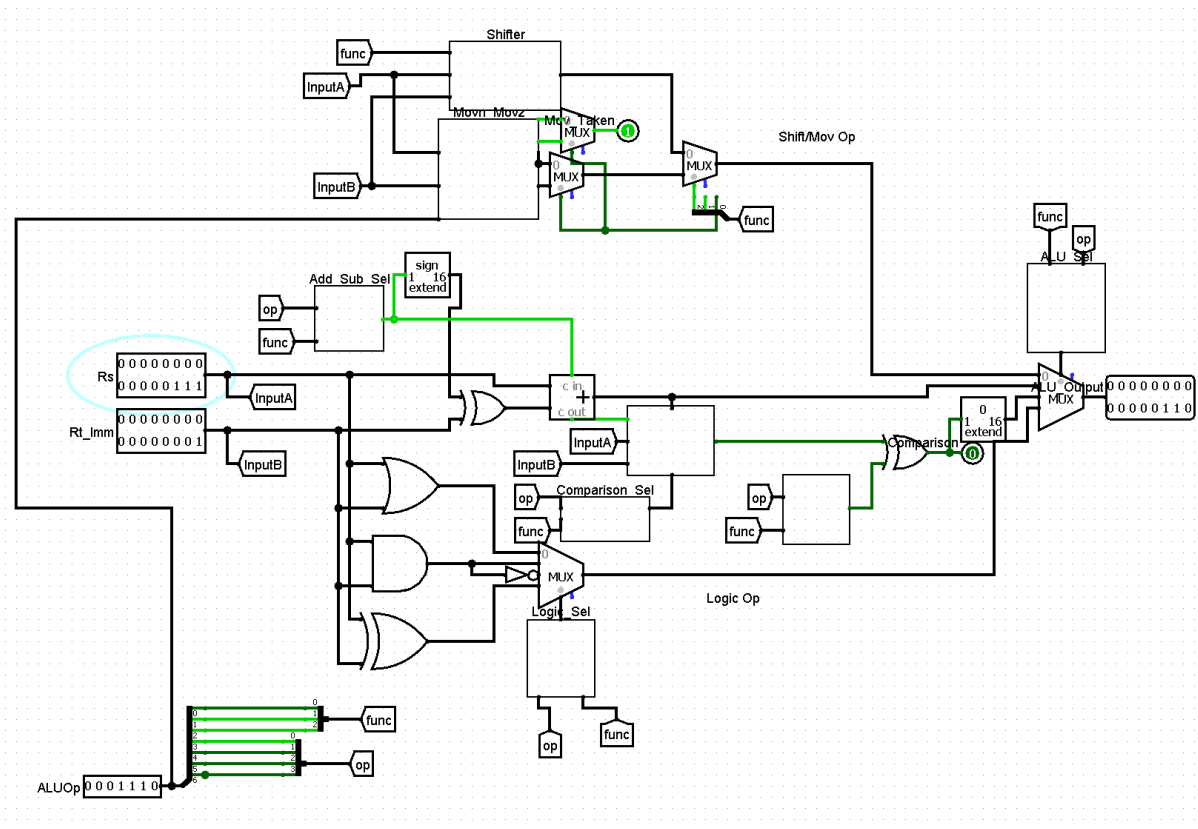
ROR:



MOVN:

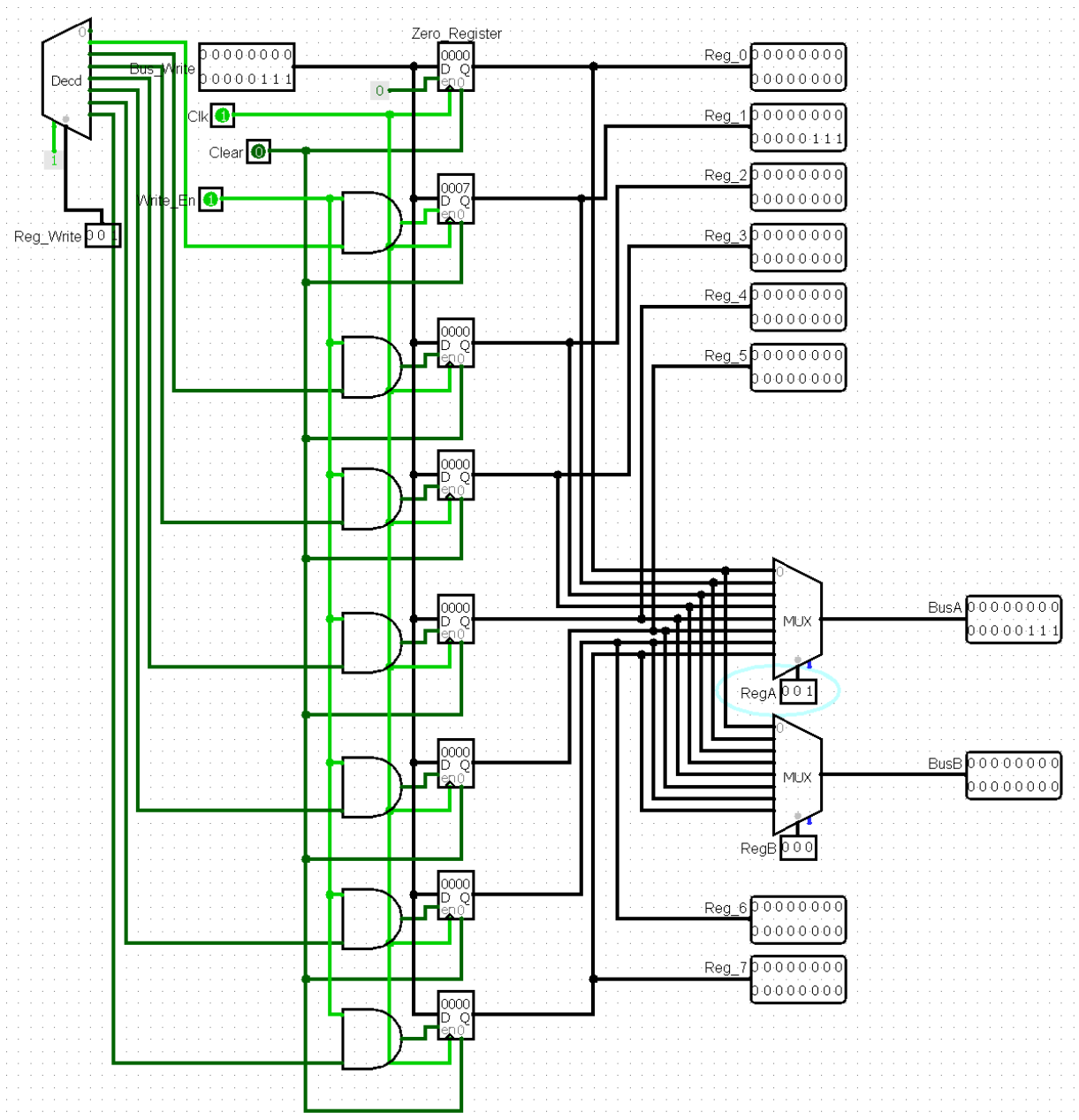


JR:



Register File

Register Read and Write:



3) Teamwork:

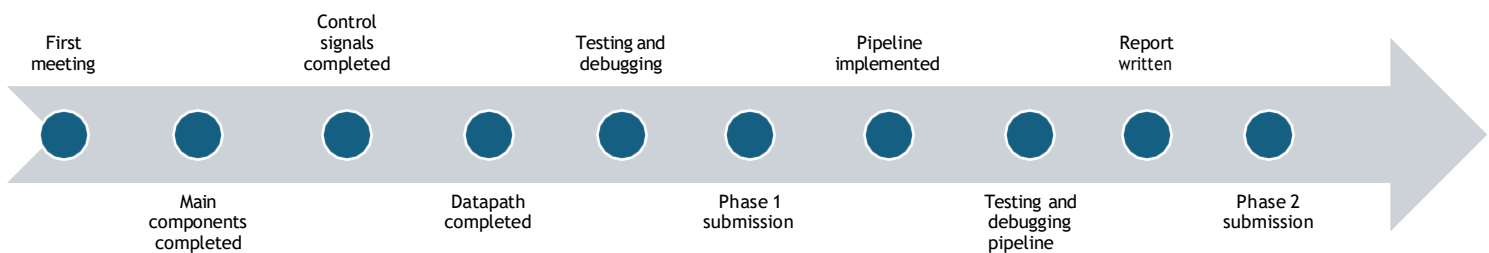
The first meeting held to discuss our approach to the project occurred during the Eid break where initial work distribution started. Another meeting was held to then discuss the work for the pipelined CPU and the report.

Both Ziad and Alaqa worked on all 3 parts: design and implementation, simulating and testing, and result reporting. Ziad's focus was more towards the design and implementation where he designed the overall CPU and developed of some components such as the ALU. Alaqa's focus was more focused on the testing and simulating where he tested the circuit and identified errors in the circuit. Both worked together during both these stages. For the report, the distribution of work is around equal.

Work Distribution

Task	Alaqa Akbar	Ziad Alalami
Design and Implementation	30%	70%
Simulating and Testing	70%	30%
Results and Report	50%	50%

Time Chart



4) Conclusion

In the end, we have learned how to design CPU components such as the ALU and Register File, and we learned how to connect the CPU components in the datapath through the control unit. Then, to optimize the throughput of the CPU, we pipelined the processor through using pipeline registers, and we dealt with hazards and dependencies through forwarding, stalling, and killing instructions. Our design might be a bit complex, but we are proud of it because we did not depend on any external source and all the design came from previous knowledge and experimentation.

